entropy

IMPACT FACTOR 2.1

Indexed in: PubMed

CITESCORE 4.9

Article

On the Security and Efficiency of TLS 1.3 Handshake with Hybrid Key Exchange from CPA-Secure KEMs

Jinrong Chen, Wei Peng, Yi Wang and Yutong Bian

MDPI

https://doi.org/10.3390/e27121242

entropy

MDPI

Article

On the Security and Efficiency of TLS 1.3 Handshake with Hybrid Key Exchange from CPA-Secure KEMs

Jinrong Chen, Wei Peng *, Yi Wang * and Yutong Bian

College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China; jinrongchen@nudt.edu.cn (J.C.); yutongbian@nudt.edu.cn (Y.B.)
* Correspondence: wpeng@nudt.edu.cn (W.P.); wangyi14@nudt.edu.cn (Y.W.)

Abstract

TLS 1.3 is a crucial protocol for securing modern internet communications. To facilitate a smooth transition to post-quantum security, hybrid key exchange, which combines classical key exchange algorithms with post-quantum key encapsulation mechanisms (KEMs), is proposed to enhance the security of the current TLS 1.3 handshake. However, existing drafts and implementations of hybrid key exchange for TLS 1.3 primarily rely on CCA-secure KEMs (i.e., secure against chosen-ciphertext attacks) based on the Fujisaki-Okamoto (FO) transform. The re-encryption step in their decapsulation algorithms not only introduces additional performance overhead but also raises the risk of side-channel attacks. Although Huguenin-Dumittan and Vaudenay (Eurocrypt 2022) and Zhou et al. (Asiacrypt 2024) demonstrated that the weaker CPA-secure KEMs (i.e., secure against chosen-plaintext attacks) suffice for constructing a secure TLS 1.3 handshake, their analyses were limited to single-KEM settings and did not consider the hybrid key exchange scenario. This work challenges the necessity of CCA security by proving that CPA-secure KEMs are sufficient for the TLS 1.3 handshake even in the hybrid key exchange setting. We provide the first formal security proofs for this claim, covering both the classical random oracle model (ROM) and the quantum random oracle model (QROM), thereby ensuring security against quantum adversaries. To validate the practical benefits, we conduct an extensive performance evaluation based on the latest OpenSSL implementation. Our results show that using CPA-secure KEMs yields up to 44.8% performance improvement at the key exchange layer and up to approximately 9% acceleration for the full TLS 1.3 handshake. Beyond performance gains, this approach reduces the codebase's attack surface by eliminating the re-encryption step, thereby mitigating a class of side-channel vulnerabilities. Our work positions CPA-secure KEMs as a secure, efficient, and practical alternative for standardizing and deploying post-quantum TLS 1.3 even with hybrid key exchange.

Keywords: TLS 1.3; hybrid key exchange; CPA-secure KEMs; post-quantum cryptography; security proof; performance evaluation; IND-CPA

check for updates

Received: 24 October 2025
Revised: 1 December 2025
Accepted: 6 December 2025
Published: 8 December 2025

Citation: Chen, J.; Peng, W.; Wang, Y.; Bian, Y. On the Security and Efficiency of TLS 1.3 Handshake with Hybrid Key Exchange from CPA-Secure KEMs. *Entropy* **2025**, *27*, 1242. https://doi.org/10.3390/e27121242

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The rapid advancement of quantum computing poses a severe threat to the security of widely deployed public-key infrastructure [1]. Shor's algorithm [2] signifies that once quantum computing technology matures, public-key cryptosystems based on traditional hardness assumptions such as the discrete logarithm [3] and integer factorization [4] problems will become insecure. TLS 1.3 [5], the predominant protocol for securing internet communication, relies on (elliptic curve) Diffie-Hellman ((EC)DH) key exchange [3,6], which

is vulnerable to Shor's algorithm. Consequently, the existing TLS 1.3 handshake protocol is not secure against adversaries with a quantum computer. To counter "harvest-now-decrypt-later" attacks, there is significant research interest in developing post-quantum TLS 1.3 handshake protocols that run on classical computers yet remain secure against quantum-capable adversaries [7–12]. A prevalent approach within the community is to replace the current (EC)DH key exchange with a post-quantum key encapsulation mechanism (KEM), which is considered an effective strategy for achieving a post-quantum TLS 1.3 handshake.

For the past, it was widely believed that the post-quantum KEM replacing (EC)DH should achieve IND-CCA (i.e., indistinguishable against chosen-ciphertext attacks) security [7,8]. In practice, for the post-quantum KEM recently standardized by the US National Institute of Standards and Technology (NIST), MLKEM [13], and the upcoming HQC candidate [14], both of them rely on the Fujisaki-Okamoto (FO) transform variant [15], which constructs the IND-CCA secure KEMs from the IND-CPA (i.e., indistinguishable against chosen-plaintext attacks) secure public-key encryption (PKE), as shown in Figure 1.

$\mathsf{Gen}(1^\lambda)$	$\mathsf{Encaps}(\mathsf{pk})$	$\mathsf{Decaps}(\mathsf{sk}, c)$
1 : $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}_{\mathsf{pke}}(1^\lambda)$	1 : $m \leftarrow\!\!\$\ \mathcal{M}_{\mathsf{pke}}$	1 : $m' := \mathsf{Dec}_{\mathsf{pke}}(\mathsf{sk}, c)$
2 : **return** $(\mathsf{pk}, \mathsf{sk})$	2 : $c := \mathsf{Enc}_{\mathsf{pke}}(\mathsf{pk}, m; G(m))$ // T_{FO}	2 : **if** $m' = \perp$ **or** $\mathsf{Enc}_{\mathsf{pke}}(\mathsf{pk}, m'; G(m')) \neq c$ **then** // T_{FO}
	3 : $c \leftarrow \mathsf{Enc}_{\mathsf{pke}}(\mathsf{pk}, m)$ // $\mathsf{T}_{\not FO}$	3 : **if** $m' = \perp$ **then** // $\mathsf{T}_{\not FO}$
	4 : $k := H(m)$	4 : **return** $\perp$
	5 : **return** (k, c)	5 : **return** $k := H(m')$

Figure 1. The FO transform $\mathsf{KEM} := \mathsf{T}_{FO}[\mathsf{PKE}, H, G]$ and its counterpart without re-encryption $\mathsf{KEM} := \mathsf{T}_{\not FO}[\mathsf{PKE}, H]$. $G : \mathcal{M}_{\mathsf{pke}} \to \mathcal{R}_{\mathsf{pke}}$ and $H : \mathcal{M}_{\mathsf{pke}} \to \mathcal{K}_{\mathsf{kem}}$ are hash functions, $\mathcal{M}_{\mathsf{pke}}$ is the message space of $\mathsf{PKE} = (\mathsf{Gen}_{\mathsf{pke}}, \mathsf{Enc}_{\mathsf{pke}}, \mathsf{Dec}_{\mathsf{pke}})$, $\mathcal{R}_{\mathsf{pke}}$ is the randomness space of $\mathsf{Enc}_{\mathsf{pke}}$, $\mathsf{Enc}_{\mathsf{pke}}(\mathsf{pk}, m; G(m))$ denotes explicitly setting the randomness used by $\mathsf{Enc}_{\mathsf{pke}}(\mathsf{pk}, m)$ to $G(m)$, and $\mathcal{K}_{\mathsf{kem}}$ is the key space of KEM. It is worth noting that the FO transform has other variants (e.g., where the key k of KEM is set as $k := H(m, c)$ instead of $H(m)$); more information is available in [15].

From Figure 1, we can see that KEM's decapsulation algorithm Decaps requires re-encryption of the decrypted message m' (i.e., line 2 in Figure 1), but this process not only introduces additional computational overhead but also increases the risk of side-channel attacks [16,17]. At Eurocrypt 2022, Huguenin-Dumittan and Vaudenay [10] were the first to demonstrate that a KEM with the weaker OW-CPA (i.e., one-way against chosen-plaintext attacks) security is sufficient for constructing a secure TLS 1.3 handshake in the random oracle model (ROM) [18]. This result was subsequently refined by Zhou et al. [11] at Asiacrypt 2024, who presented tighter security proofs in the ROM based on both OW-CPA and IND-CPA secure KEMs and, crucially, extended the results to the quantum random oracle model (QROM) [19], which further confirms that CPA-secure post-quantum KEMs are adequate for building secure post-quantum TLS 1.3 handshakes. As evident from the construction of CPA-secure KEMs without re-encryption [20] (see Figure 1), the elimination of re-encryption in the decapsulation algorithm reduces computational cost and mitigates the aforementioned side-channel risks.

However, due to ongoing scrutiny of post-quantum KEM security and compatibility concerns, hybrid key exchange schemes, which combine post-quantum KEMs with well-established classical key exchange (e.g., (EC)DH), represent a pragmatic strategy for a smooth transition to post-quantum TLS 1.3. Note that the IETF TLS Working Group is

standardizing hybrid key exchange for TLS 1.3 [21,22], with a representative structure depicted in Figure 2 (where (EC)DH is regarded as a KEM as described in [10]).

$\mathsf{Gen}(1^\lambda)$	$\mathsf{Encaps}(\mathsf{pk})$	$\mathsf{Decaps}(\mathsf{sk}, c)$
1 : $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{Gen}_1(1^\lambda)$	1 : $(\mathsf{pk}_1, \mathsf{pk}_2) := \mathsf{pk}$	1 : $(\mathsf{sk}_1, \mathsf{sk}_2) := \mathsf{sk},\ (c_1, c_2) := c$
$(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$	2 : $(k_1', c_1) \leftarrow \mathsf{Encaps}_1(\mathsf{pk}_1)$	2 : $k_1' := \mathsf{Decaps}_1(\mathsf{sk}_1, c_1)$
2 : $\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$	$(k_2', c_2) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$	$k_2' := \mathsf{Decaps}_2(\mathsf{sk}_2, c_2)$
$\mathsf{sk} := (\mathsf{sk}_1, \mathsf{sk}_2)$	3 : $k := (k_1', k_2')$	3 : **if** $\perp \in \{k_1', k_2'\}$ **then**
3 : **return** $(\mathsf{pk}, \mathsf{sk})$	$c := (c_1, c_2)$	**return** $\perp$
	4 : **return** (k, c)	4 : **return** $k := (k_1', k_2')$

Figure 2. $\mathsf{KEM} := \mathsf{Comb}[\mathsf{KEM}_1, \mathsf{KEM}_2]$. Summarized from [21,22].

This hybrid approach essentially employs a simple "concatenation" method, offering the advantage of not requiring additional cryptographic primitives or significant extra computation. It is important to note that the works of Huguenin-Dumittan [10] and Vaudenay and Zhou et al. [11] focus solely on the security of TLS 1.3 using a *single* KEM and do not address the hybrid setting. Critically, the IND-CPA security of individual KEM components does not automatically guarantee the IND-CPA security of the combined KEM. Considering Figure 2, if KEM_2 is not IND-CPA secure, there exists an adversary $\mathcal{A}$ could effectively distinguish whether k_2' in (k_2', c_2) is the real key or a random key from $\mathcal{K}_{\mathrm{kem}_2}$. Then, $\mathcal{A}$ could break the IND-CPA security of the combined KEM by using (k_2', c_2) from the challenge key and ciphertext $k = (k_1', k_2')$ and $c = (c_1, c_2)$. Therefore, whether CPA-secure KEMs remain sufficient for constructing a secure TLS 1.3 handshake in the context of hybrid key exchange remains an open problem.

1.1. Our Contributions

In this work, we presents the first security analysis of the TLS 1.3 handshake using CPA-secure KEMs within hybrid key exchange setting. Specifically, our contributions are as follows:

1. We provide the first security proofs for the TLS 1.3 handshake based on CPA-secure KEMs in a hybrid key exchange scenario, both in the ROM and QROM. This formally establishes that CPA-secure KEMs are indeed sufficient for constructing secure post-quantum TLS 1.3 handshakes even when combined with classical key exchange.
2. We implement and evaluate the performance of the hybrid schemes specified in the IETF draft [21], i.e., X25519MLKEM768, SecP256r1MLKEM768, and SecP384-MLKEM1024, using the latest OpenSSL (https://github.com/openssl/openssl/releases/download/openssl-3.6.0/openssl-3.6.0.tar.gz accessed on 23 October 2025). Our experimental results compare the efficiency when the underlying base KEM provides IND-CCA security versus IND-CPA security, and demonstrate that at the key exchange layer, using a CPA-secure KEM can yield a performance improvement of up to approximately 44.8%.
3. Finally, we benchmark the overall TLS 1.3 handshake performance in the hybrid setting for both IND-CCA and IND-CPA secure underlying base KEMs. The experimental results show that at the full protocol level, the efficiency gain from using CPA-secure KEMs is more modest, not exceeding 9%. Nonetheless, employing CPA-secure KEMs still reduces the protocol's vulnerability to side-channel attacks. Furthermore, eliminating the need for extra hash function calls (e.g., in re-encryption) can potentially save "hundreds of thousands or even millions of dollars a year" in operational costs [23].

In summary, this work provides a comprehensive analysis of both the security and efficiency of the TLS 1.3 handshake with hybrid key exchange from CPA-secure KEMs. Our findings offer additional options and support for the ongoing standardization of post-quantum TLS 1.3.

1.2. Technique Overview

The KEM-based TLS 1.3 handshake protocol is depicted in Figure 3. During the key exchange, the client first computes $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KEM.Gen}(1^\lambda)$ and sends pk to the server (as the client share). Upon receiving pk, the server computes $(k, c) \leftarrow \mathsf{KEM.Encaps}(\mathsf{pk})$ and sends c to the client (as the server share). Finally, the client computes $k := \mathsf{KEM.Decaps}(\mathsf{sk}, c)$ upon receiving c. Both parties then use k as the shared secret.

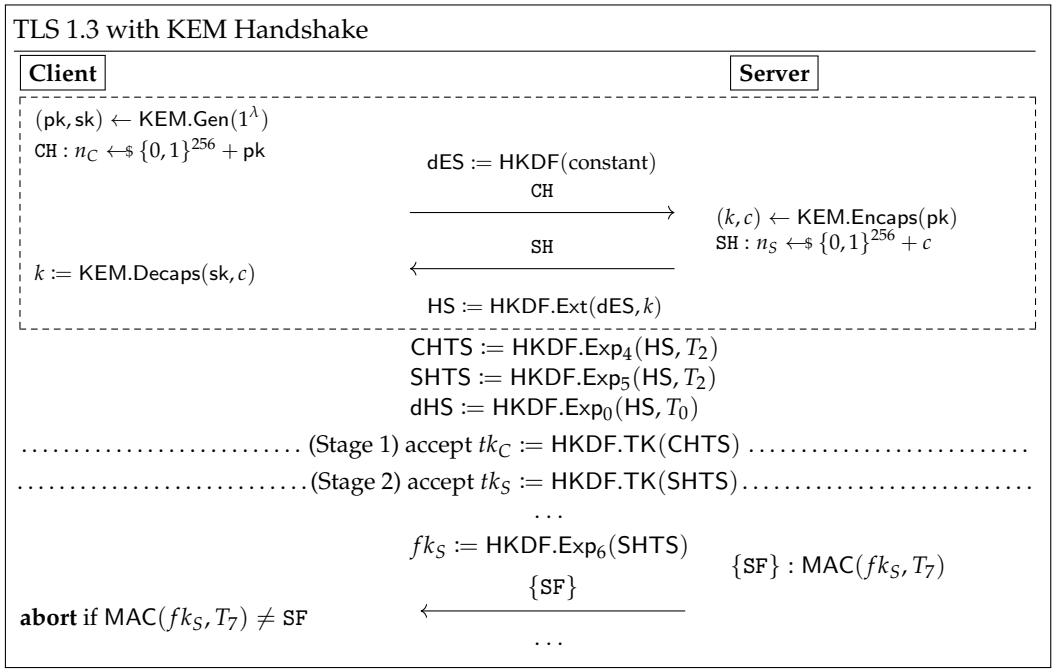


Figure 3. TLS 1.3 handshake with KEM, adapted from [10]. $\{\cdot\}$ denotes messages encrypted using tk_S, and T_i is the hash of the transcript up to the i-th message. For brevity, `ClientHello` and `ClientKeyShare` are merged into `CH`, while `ServerHello` and `ServerKeyShare` are merged into `SH`. Only some important steps are shown. The remaining stage keys (3-6, omitted) are derived from dHS.

1.2.1. Security Proof of TLS 1.3 Handshake with Hybrid Key Exchange from CPA-Secure KEMs

Analyzing the security of TLS 1.3 handshake with hybrid key exchange is non-trivial. The protocol comprises numerous cryptographic primitives (e.g., key derivation function HKDF, message authentication code MAC, etc.), intertwined in a manner where their security properties mutually influence each other, collectively forming the complex TLS 1.3 protocol. Fortunately, recent work by Chen et al. [12] introduces a modular analysis framework. They conceptualize the component within the dashed box in Figure 3 as a new KEM, termed the *intermediate* KEM (see Figure 4).

From this perspective, the client and the server can be viewed as using this *intermediate* KEM to negotiate HS as their "shared secret" following the aforementioned process. Based on the *intermediate* KEM, Chen et al. [12] proposed a modular analysis approach, as illustrated in the section delineated by the dashed box in Figure 5.

$\mathsf{Gen}_{int.}(1^\lambda)$	$\mathsf{Encaps}_{int.}(\mathsf{pk})$	$\mathsf{Decaps}_{int.}(\mathsf{sk}, c)$
1 : $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$	1 : $(k', c) \leftarrow \mathsf{Encaps}(\mathsf{pk})$	1 : $k' := \mathsf{Decaps}(\mathsf{sk}, c)$
2 : **return** $(\mathsf{pk}, \mathsf{sk})$	2 : $k := G(k')$	2 : **if** $k' = \perp$ **then** : **return** $\perp$
	3 : **return** (k, c)	3 : **return** $k := G(k')$

Figure 4. The *intermediate* KEM $\mathsf{KEM}_{int.} := \mathsf{T}_{int.}[\mathsf{KEM}, G]$, adapted from [12], where G is a key derivation function (corresponding to $\mathsf{HKDF.Ext}(\mathsf{dES}, \cdot)$ in Figure 3).

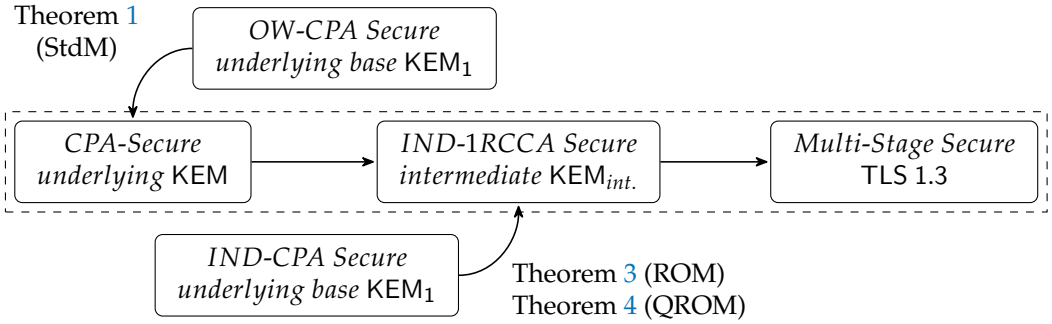


Figure 5. Modular analysis of TLS 1.3 handshake. $\boxed{\text{A}} \rightarrow \boxed{\text{B}}$ denotes that A implies B provided the other cryptographic components are secure. Results inside the dashed box were established by Chen et al. [12], while those outside it are contributions of this work. The notes following the theorems indicate the proof model, where StdM denotes the (random oracle independent) standard model.

Briefly, Chen et al. [12] prove that the OW-CPA or IND-CPA security of the underlying KEM implies the IND-1RCCA security of the *intermediate* KEM, and that IND-1RCCA security of the *intermediate* KEM suffices to guarantee the overall security of the TLS 1.3 handshake. Although Chen et al. [12] focused on single KEM instances, their modular approach can be extended to the hybrid key exchange scenario. When TLS 1.3 employs hybrid key exchange, the underlying KEM in Figure 5 is constructed by combining several *base KEMs* according to the method shown in Figure 2. This modular analysis simplifies the security analysis of the TLS 1.3 handshake, allowing us to concentrate our security focus primarily on the KEM's security.

Conclusion 1: When one base KEM satisfies OW-CPA security (Section 3.1). The security analysis for the TLS 1.3 handshake in this case is relatively straightforward. We need only ensure that the underlying hybrid KEM (in Figure 5) is OW-CPA secure as a whole. This can be proven by contradiction: if there exists a PPT adversary $\mathcal{A}$ that can break the hybrid KEM, we can construct a new PPT adversary $\mathcal{B}$ from $\mathcal{A}$ that breaks the OW-CPA security of any constituent base KEM. Given that at least one base KEM is OW-CPA secure, such an adversary $\mathcal{A}$ cannot exist, thereby proving the OW-CPA security of the hybrid KEM (see Theorem 1). Once the hybrid KEM's OW-CPA security is established, combining this with Chen et al.'s result [12] proves that the TLS 1.3 handshake based on hybrid key exchange is secure as long as one base KEM is OW-CPA secure (see Theorem 2).

Conclusion 2: When one base KEM satisfies IND-CPA security (Section 3.2). The security analysis here is slightly more complex. As discussed in the paragraph following Figure 2, the IND-CPA security of a single base KEM does *not* necessarily imply the IND-CPA security of the hybrid KEM. Therefore, we cannot rely on a simple analysis of the hybrid KEM's security as in the OW-CPA case. Fortunately, we can shift our focus to the IND-1RCCA security of the *intermediate* KEM. If we can prove that the IND-CPA security of one base KEM implies the IND-1RCCA security of the *intermediate* KEM, then, combined with Chen et al.'s results [12], we can conclude that the TLS 1.3 handshake with hybrid key

exchange is secure (Theorem 5). Depending on the adversary's capabilities, we provide security proofs in both the ROM (Theorem 3) and the QROM (Theorem 4).

CASE 1: THE PROOF OF THE KEM IN THE ROM (Section 3.2.1). The key to reducing the IND-1RCCA security of the *intermediate* KEM to the IND-CPA security of a base KEM lies in simulating the decapsulation oracle in the IND-1RCCA security game without access to the private key sk. We consider three cases based on the value $k' := \mathsf{Decaps}(\mathsf{sk}, c)$ computed by the decapsulation oracle:

1. When $k' = k^*$ (where $G(k^*)$ is the challenge key), the actual decapsulation result $G(k')$ must equal the challenge key, so the oracle returns `test`.
2. When $k' = \perp$, the oracle returns $\perp$, consistent with the actual decapsulation algorithm.
3. For other values of k', the oracle must query the random oracle G on k' to determine the return value. If the adversary has not queried k' itself, the return value is random and independent from the adversary's view. Consequently, in this case, the oracle can use a *pre-selected* random value to determine its return value. When handling the adversary's random oracle queries, the reduction can randomly select one query (or with some probability, none) to respond with this pre-selected value. This achieves the goal of determining the oracle's return value without needing to use sk to compute k'.

Thus, the reduction algorithm can guess which of the three cases applies for the single decapsulation query (succeeding with probability 1/3), enabling it to simulate the IND-1RCCA decapsulation oracle effectively without using sk to compute k'.

CASE 2: THE PROOF OF THE KEM IN THE QROM (Section 3.2.2). When considering the quantum-capable adversaries, we should extend the security proof to the QROM. The core challenge remains simulating the IND-1RCCA decapsulation oracle without sk. We can still employ random guessing for the decapsulation result: if $k' = k^*$ is guessed, the oracle returns `test`; if $k' = \perp$ is guessed, it returns $\perp$. The key difference arises in the remaining case. Due to fundamental quantum principles (specifically, the no-cloning principle), we cannot simply record the adversary's "quantum" queries without significantly disturbing its state, as was possible in the ROM. However, we can utilize the *singular-classical-query measure-and-reprogram lemma* [24] to deal with this problem. This technique employs a measure-and-reprogram approach for the random oracle, allowing for simulation of the decapsulation oracle without sk and enabling the security proof in the QROM.

1.2.2. Efficiency Analysis of TLS 1.3 Handshake with Hybrid Key Exchange from CPA-Secure KEMs

We conduct an experimental performance analysis of both the hybrid key exchange itself and the full TLS 1.3 handshake with hybrid key exchange, based on OpenSSL (https://openssl-library.org/ accessed on 23 October 2025), a widely-used cryptography library. The latest OpenSSL version supports three hybrid key exchange methods that are specified in the IETF draft [21]: X25519MLKEM768, SecP256r1MLKEM768, and SecP384r1MLKEM1024. The MLKEM implementation follows NIST FIPS 203 standard [13] and provides IND-CCA security.

EXPERIMENTAL ANALYSIS AT THE KEY EXCHANGE LEVEL (Section 4.1). We modify the OpenSSL source code related to MLKEM, specifically altering its encapsulation and decapsulation algorithms according to ([20], Figure 6) (denoted as $\mathsf{T}_{\not{f}O}$ in this work and shown in Figure 1) to effectively reduce its security to IND-CPA. After compiling them, we use the `openssl speed` tool to obtain performance data for the hybrid key exchange, allowing us to analyze the performance improvement gained by removing re-encryption.

EXPERIMENTAL ANALYSIS AT THE PROTOCOL LEVEL (Section 4.2). For protocol-level testing of TLS 1.3, we simulate the handshake between a client and a server locally using two terminals, thereby eliminating external network latency and focusing solely on protocol

efficiency. Specifically, we start one terminal to act as the server (after generating its certificate) using the `openssl s_server` tool. Then, we use another terminal with the `openssl s_time` tool, acting as the client, to measure the handshake time with the server. However, there are two practical issues: first, `openssl s_time` does not allow for specification of the key exchange algorithm for the handshake; second, the default list of key exchange algorithms supported by `openssl s_time` does not include all the hybrid schemes.

To address the first issue, we can use the `-groups` option of `openssl s_server` to restrict the server to support only one specific key exchange method, thereby forcing `openssl s_time` to use that method. For the second issue, we modify the OpenSSL configuration file (`openssl.cnf`) to include all the hybrid schemes in OpenSSL's default key exchange list. With this setup, we establish a local test environment for TLS 1.3 using different hybrid key exchange schemes, enabling further analysis of the performance benefits of using weaker-security KEMs at the protocol level.

1.3. Related Work

Since the formal standardization of TLS 1.3 [5], numerous researchers have investigated its security [25–27]. Regarding the handshake protocol specifically, Dowling et al. [8] extended the analysis of earlier TLS 1.3 drafts [28–32]. They formally defined the multi-stage key exchange model to comprehensively capture the properties of keys derived in different stages of the TLS 1.3 handshake and provided a formal security proof in the standard model based on the PRF-ODH assumption [33]. As research progressed, many began exploring the integration of KEMs into TLS 1.3 to enhance its security [7–12]. For KEMs intended to replace (EC)DH key exchange, much prior work [7,8] assumed that IND-CCA security was necessary. However, Huguenin-Dumittan and Vaudenay [10] were the first to prove, in the ROM, that OW-CPA secure KEMs are sufficient for the TLS 1.3 handshake's security, concurrently showing that the CDH assumption [3] suffices for proving the security of the existing TLS 1.3. Zhou et al. [11] subsequently improved the security analyses by Huguenin-Dumittan and Vaudenay [10] in the ROM and, crucially, extended the result to the QROM, demonstrating that CPA-secure KEMs also suffice for post-quantum TLS 1.3 security.

Note that the aforementioned studies primarily focus on TLS 1.3 security with a *single* KEM and do not address the impact of *hybrid* key exchange. In practice, however, many institutions like Google and Cloudflare are gradually deploying hybrid key exchange in TLS 1.3 (https://www.intelligentliving.co/quantum-hybrid-tls-ml-kem-browser/ accessed on 23 October 2025), combining classical key exchange with post-quantum KEMs, to facilitate a smooth transition from classical to post-quantum security. This trend is accompanied by several IETF drafts proposing methods for hybrid key exchange in TLS [21,22]. Researchers have also extensively studied various hybrid key exchange methods [34–36], and recent work by Liu et al. [37] unified several primary approaches. Nevertheless, these studies did not specifically consider the security implications of using CPA-secure KEMs within the hybrid key exchange framework for the TLS 1.3 handshake.

Regarding benchmarking the post-quantum TLS 1.3 handshake with hybrid key exchange, Zheng et al. [38] provided a detailed analysis of various hybrid schemes based on ML-KEM. However, the ML-KEM they considered provided the stronger IND-1CCA security, not IND-CPA security. Benčina et al. [20] investigated hybrid key exchange using CPA-secure KEMs for the SSH protocol. They analyzed the performance of hybrid key exchange combining X25519 with both CCA-secure and CPA-secure variants of ML-KEM and Streamlined NTRU Prime, as well as the overall SSH protocol performance. However, their findings are not directly applicable to the TLS 1.3 handshake with hybrid key exchange.

1.4. Paper Structure

We review necessary preliminaries in Section 2. Section 3 provides the security proofs for TLS 1.3 handshake with hybrid key exchange from CPA-secure KEMs. Section 4 reports our experimental results for both the key exchange level and the protocol level under hybrid key exchange conditions. Finally, Section 5 concludes the paper.

2. Preliminaries

This section reviews some cryptographic primitives and relevant lemmas.

2.1. Notation

Let $H : \mathcal{X} \to \mathcal{Y}$ denote the function with domain $\mathcal{X}$ and codomain $\mathcal{Y}$, and Ω_H be the set of all such functions. For a set $\mathcal{S}$, its size is denoted by $|\mathcal{S}|$, and sampling an element s uniformly at random from $\mathcal{S}$ is written as $s \leftarrow\!\!\$\ \mathcal{S}$. For a probabilistic (resp. deterministic) algorithm A with input x, we denote its output as y by $y \leftarrow A(x)$ (resp. $y := A(x)$). An oracle algorithm A with classical (resp. quantum) access to an oracle H is denoted by A^H (resp. $A^{|H\rangle}$). The symbol $[x = y]$ represents an indicator function that equals 1 if $x = y$ and 0 otherwise. The security parameter is denoted by λ, and probabilistic polynomial time is abbreviated as PPT.

2.2. Cryptographic Primitives

Definition 1 (Key Encapsulation Mechanism, KEM)**.** *A Key Encapsulation Mechanism (KEM)* KEM *consists of three* PPT *algorithms, parameterized by a security parameter λ, a key space $\mathcal{K}$, and a ciphertext space $\mathcal{C}$, where: (1) The key generation algorithm* Gen *is a probabilistic algorithm that takes 1^λ as input and outputs a public key/secret key pair $(\mathsf{pk}, \mathsf{sk})$; (2) The encapsulation algorithm* Encaps *is a probabilistic algorithm that takes* pk *as input and outputs a key-ciphertext pair $(k, c) \in \mathcal{K} \times \mathcal{C}$; (3) The decapsulation algorithm* Decaps *is a deterministic algorithm that takes (sk, c) as input and outputs $k \in \mathcal{K} \cup \{\perp\}$. We say* $\mathsf{KEM} = (\mathsf{Gen}, \mathsf{Encaps}, \mathsf{Decaps})$ *is defined over $(\mathcal{K}, \mathcal{C})$.*

The correctness requirement for KEM *is that for all* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$ *and all* $(k, c) \leftarrow \mathsf{Encaps}(\mathsf{pk})$*, it holds that* $\mathsf{Decaps}(\mathsf{sk}, c) = k$*. If $\perp \notin \mathcal{K}$,* KEM *is said to have explicit rejection; if $\perp$ is a random value in $\mathcal{K}$,* KEM *is said to have implicit rejection.*

Definition 2 (Security of KEM)**.** *The* OW-CPA *security and* IND-ATK *security (where* $\mathsf{ATK} \in \{\mathsf{CPA}, \mathsf{1RCCA}, \mathsf{1CCA}, \mathsf{CCA}\}$*) of a KEM* $\mathsf{KEM} = (\mathsf{Gen}, \mathsf{Encaps}, \mathsf{Decaps})$ *defined over $(\mathcal{K}, \mathcal{C})$ are defined through a game played between a challenger and an adversary $\mathcal{A}$, as shown in Figure 6.*

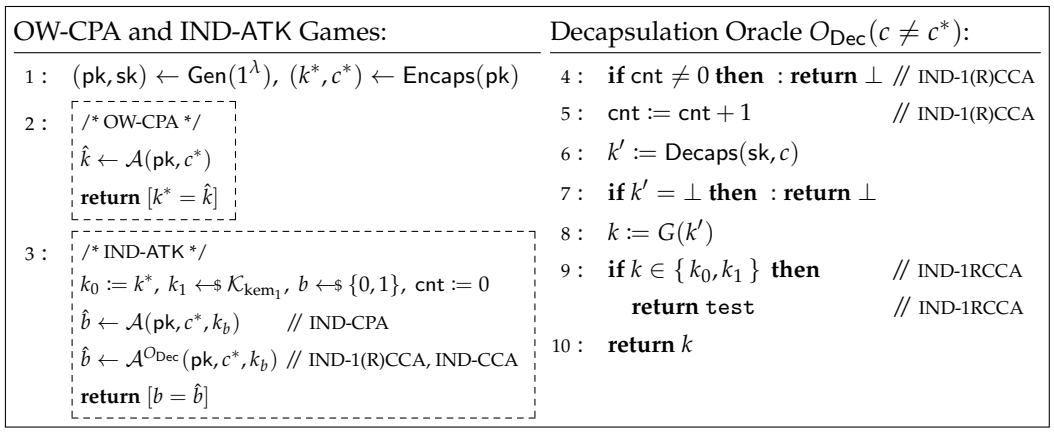


Figure 6. Security games for OW-CPA and IND-ATK (where $\mathsf{ATK} \in \{\mathsf{CPA}, \mathsf{1RCCA}, \mathsf{1CCA}, \mathsf{CCA}\}$) security of $\mathsf{KEM} = (\mathsf{Gen}, \mathsf{Encaps}, \mathsf{Decaps})$.

We define the advantage of $\mathcal{A}$ against the OW-CPA *security of* KEM *as* $\mathsf{Adv}^{\text{OW-CPA}}_{\mathsf{KEM}}(\mathcal{A}) := \Pr[k^* = \hat{k}]$, *and the advantage against the* IND-ATK *security of* KEM *as* $\mathsf{Adv}^{\text{IND-ATK}}_{\mathsf{KEM}}(\mathcal{A}) := \left|\Pr[b = \hat{b}] - 1/2\right|$. *If for any* PPT *adversary* $\mathcal{A}$, $\mathsf{Adv}^{\text{OW-CPA}}_{\mathsf{KEM}}(\mathcal{A})$ *(or* $\mathsf{Adv}^{\text{IND-ATK}}_{\mathsf{KEM}}(\mathcal{A})$*) is negligible, then* KEM *is said to be* OW-CPA *secure (or* IND-ATK *secure, respectively). The values* k^*, k_0, k_1 *sent to* $\mathcal{A}$ *are referred to as the challenge keys, and* c *is referred to as the challenge ciphertext. Sometimes, to distinguish between* k_0 *and* k_1*, we also call* k_0 *the real key and* k_1 *the random key.*

2.3. (Quantum) Random Oracle Model

The random oracle model (ROM), introduced by Bellare and Rogaway [18], is an important cryptographic model that treats hash functions as ideal public random oracles. In this model, an adversary can only obtain the value of a hash function $H(x)$ by querying the random oracle H on x; otherwise, $H(x)$ remains independent and random from the adversary's perspective.

The quantum random oracle Model (QROM) [19] extends the ROM by allowing adversaries to query the random oracle in quantum superposition states. In QROM, the random oracle H is modeled as a unitary operator $U_H : |x, y\rangle \mapsto |x, y \oplus H(x)\rangle$, where the quantum registers containing x and y are referred to as the input and output registers (of this random oracle H query), respectively. This model is crucial for analyzing the security of cryptographic schemes against quantum-capable adversaries, as it captures the adversary's quantum computational power. For further background on quantum computing, readers may refer to [39].

2.4. Important Lemmas

Lemma 1 (Double-sided one-way to hiding, DS-O2H ([40] Lemma 5), adapted)**.** *Let* $G, H : \mathcal{X} \to \mathcal{Y}$ *be random functions such that* $\forall x \neq x^* \in \mathcal{X}$, $G(x) = H(x)$, *and let* z *be a random value, where* (G, H, x^*, z) *may have an arbitrary joint distribution. Let* $A^{|H\rangle}(z)$ *be an oracle algorithm (not necessarily unitary) with quantum access to random oracle* H*. Then there exists a double-sided oracle algorithm* $B^{|G\rangle,|H\rangle}(z)$ *with quantum access to both random oracles* G *and* H*, such that*

$$\left|\Pr[1 \leftarrow A^{|G\rangle}(z)] - \Pr[1 \leftarrow A^{|H\rangle}(z)]\right| \leq 2\sqrt{\Pr[x^* = \hat{x} : \hat{x} \leftarrow B^{|G\rangle,|H\rangle}(z)]}\,.$$

Lemma 2 (Search in double-sided oracle ([24] Lemma 2.3), adapted)**.** *In Lemma 1, if* $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2$, $x^* = (x_1, x_2) \in \mathcal{X}_1 \times \mathcal{X}_2$, *and* $x_1 \in \mathcal{X}_1$ *is independent of* G *(or* H*) and* z*, then*

$$\Pr[x^* = \hat{x} : \hat{x} \leftarrow B^{|G\rangle,|H\rangle}(z)] \leq q_G^2/|\mathcal{X}_1| \text{ (or } q_H^2/|\mathcal{X}_1|)\,,$$

where q_G *(or* q_H*) is the bounded number of* G *(or* H*) random oracle queries made by* $A^{|G\rangle}(z)$ *(or* $A^{|H\rangle}(z)$*).*

Lemma 3 (Singular-classical-query measure-and-reprogram lemma ([24] Lemma 3.1), adapted)**.** *Let* $A^{|H\rangle}$ *be a quantum oracle algorithm making at most* q *(quantum) queries to the random oracle* $H : \mathcal{X} \to \mathcal{Y}$*, and outputting some classical* $x \in \mathcal{X}$ *and a (possibly quantum) output* z*. In particular, the input state of the* i^**-th query of* $A^{|H\rangle}$ *is* $|x\rangle$*, where* $i^* \in \{1, \ldots, q\}$*, and* $|x\rangle$ *is a classical state identical to the* x *output by* $A^{|H\rangle}$*.*

Let $S^A(\Theta)$ *be a quantum oracle algorithm that picks a random pair* $(i, b_0) \leftarrow\!\$\ (\{1, \ldots, q\} \setminus \{i^*\} \times \{0, 1\}) \cup \{(q+1, 0)\}$*, runs* $A^{|H_i^{i^*}\rangle}$ *to output* z*, where* $H_i^{i^*}$ *is an oracle that returns* Θ *for the* i^**-th* H *query of* A*, measures the input of the* i*-th* H *query of* A *to obtain* x*, uses* H *to answer the* l*-th* H *query of* A *(for* $l < (i + b_0)$ *and* $l \neq i^*$*), and uses* $H_{x\Theta}$ *(where* $H_{x\Theta}(x) = \Theta$ *and* $H_{x\Theta}(x') = H(x')$ *for all* $x' \neq x$*) to answer the* l*-th* H *query of* A *(for* $l \geq (i + b_0)$ *and* $l \neq i^*$*).*

Let $S_1^A(\Theta)$ *be a quantum oracle algorithm that picks a random pair* $(j, b_1) \in (\{i^* + 1, \ldots, q\} \times \{0, 1\}) \cup \{(q+1, 0)\}$*, runs* $A^{|H_j\rangle}$ *to output* z*, where* H_j *is an oracle that measures*

the input of the j-th H query of A to obtain x, uses H to answer the l-th oracle query of A (for $l < (j + b_1)$), and uses $H_{x\Theta}$ to answer the l-th oracle query of A (for $l \geq (j + b_1)$).

Then for any $x_0 \in \mathcal{X}$, $i^* \in \{1, \ldots, q\}$, and any predicate V, we have

$$\begin{aligned}\Pr_H[x = x_0 \wedge V(x, H(x), z) = 1 : (x, z) \leftarrow A^{|H\rangle}] \\ \leq 2(2q-1)^2 \Pr_{H,\Theta}[x = x_0 \wedge V(x, \Theta, z) = 1 : (x, z) \leftarrow S^A] \\ + 8q^2 \Pr_{H,\Theta}[x = x_0 \wedge V(x, \Theta, z) = 1 : (x, z) \leftarrow S_1^A] ,\end{aligned}$$

where the subscripts in $\Pr_H$ and $\Pr_{H,\Theta}$ indicate that the probability is taken over the random choices of H and Θ. Furthermore, if $V = V_1 \wedge V_2$ and $V_1(x, y, z) = 1$ if and only if y is returned in the response to the i^*-th H-query of A, then

$$\sum_{x_0} \Pr_{H,\Theta}[x = x_0 \wedge V(x, \Theta, z) = 1 : (x, z) \leftarrow S_1^A] \leq 1/|\mathcal{Y}| .$$

Lemma 4 (Difference lemma [41])**.** *Let W_1, W_2, and Z be events defined in some probability space, and let $\bar{Z}$ denote the complement of Z. If $\Pr[W_1 \wedge \bar{Z}] = \Pr[W_2 \wedge \bar{Z}]$, then*

$$|\Pr[W_2] - \Pr[W_1]| \leq \Pr[Z] .$$

3. Security of TLS 1.3 Hybrid Handshake

We now prove that CPA-secure KEMs suffice for the TLS 1.3 hybrid handshake, considering both OW-CPA and IND-CPA-secure base KEMs.

3.1. When the Underlying Base KEM Is OW-CPA *Secure*

This case is straightforward. Following the modular analysis (Figure 5), we first prove that the hybrid KEM inherits OW-CPA security if one base KEM is OW-CPA secure (Theorem 1). Then, applying results from [12], we finally establish the security of the TLS 1.3 hybrid handshake from OW-CPA secure KEMs (Theorem 2).

Theorem 1 (OW-CPA security of the hybrid KEM)**.** *If* KEM_1 *is* OW-CPA *secure, then* $\mathsf{KEM} := \mathsf{Comb}[\mathsf{KEM}_1, \mathsf{KEM}_2]$ *(as shown in Figure 2) is also* OW-CPA *secure. Specifically, for any* PPT *adversary* $\mathcal{A}$ *against* KEM*, there exists a* PPT *adversary* $\mathcal{B}$ *against* KEM_1*, such that*

$$\mathsf{Adv}^{\text{OW-CPA}}_{\mathsf{KEM}}(\mathcal{A}) \leq \mathsf{Adv}^{\text{OW-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) .$$

Proof of Theorem 1. The proof constructs an adversary $\mathcal{B}$ against KEM_1 by using adversary $\mathcal{A}$. Given challenge (pk_1, c_1) from its OW-CPA challenger, $\mathcal{B}$ generates $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$ and $(k_2', c_2) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$, then forms the hybrid challenge $\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$, $c := (c_1, c_2)$ for $\mathcal{A}$. Feeds (pk, c) to $\mathcal{A}$. When $\mathcal{A}$ outputs a solution $k = (k_1, k_2)$ for KEM, $\mathcal{B}$ returns k_1 as the solution for KEM_1. The validity of k_1 follows directly from the winning condition of $\mathcal{A}$. Therefore, we have

$$\mathsf{Adv}^{\text{OW-CPA}}_{\mathsf{KEM}}(\mathcal{A}) \leq \mathsf{Adv}^{\text{OW-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) .$$

□

Combining with relevant results from [12], we can obtain the security of TLS 1.3 handshake with hybrid key exchange from OW-CPA secure base KEMs.

Theorem 2 (Security of TLS 1.3 handshake with hybrid key exchange from OW-CPA secure KEM)**.** *If the other components in TLS 1.3 handshake are secure (as defined in [12]), then the entire*

TLS 1.3 handshake with hybrid key exchange is secure, provided that at least one underlying base KEM is OW-CPA *secure.*

Proof of Theorem 2. This result follows directly from Theorem 1, combined with Theorems 4.1 and 4.2 of [12] (which show that IND-1RCCA security of the *intermediate* KEM implies a special IND-1CCA-1MAC security), Theorem 5.1 (which establishes that OW-CPA security of the underlying KEM implies IND-1RCCA security of the *intermediate* KEM), and Theorems 6.2 and 6.3 (which prove that IND-1CCA-1MAC security of the *intermediate* KEM implies the multi-stage security of the TLS 1.3 handshake) in the same reference. □

3.2. When the Underlying Base KEM Is IND-CPA *Secure*

Here, we first prove the IND-1RCCA security of the *intermediate* KEM in the ROM and QROM (Theorems 3 and 4), and then derive the security of the full handshake (Theorem 5).

3.2.1. Security of the *Intermediate* KEM in the ROM

Theorem 3 (IND-1RCCA security of the hybrid $\mathsf{KEM}_{int.}$ in the ROM)**.** *Let* $\mathsf{KEM}_{int.} := \mathsf{T}_{int.}[\mathsf{Comb}[\mathsf{KEM}_1, \mathsf{KEM}_2], G]$ *(*Comb *and* $\mathsf{T}_{int.}$ *defined in Figures 2 and 4), with G modeled as a random oracle. If* KEM_1 *is* IND-CPA *secure, then* $\mathsf{KEM}_{int.}$ *is* IND-1RCCA *secure. Specifically, for any* PPT *adversary* $\mathcal{A}$ *against* $\mathsf{KEM}_{int.}$*, there exists a* PPT *adversary* $\mathcal{B}$ *against* KEM_1*, such that*

$$\mathsf{Adv}^{\text{IND-1RCCA}}_{\mathsf{KEM}_{int.}}(\mathcal{A}) \leq 6(q_G+1)\cdot \mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) + 3q_G(q_G+1)/\left|\mathcal{K}_{\mathsf{kem}_1}\right|,$$

where q_G *is the number of queries to G made by* $\mathcal{A}$*, and* $\mathcal{K}_{\mathsf{KEM}_1}$ *is the key space of* KEM_1*.*

Proof of Theorem 3. We define a sequence of games G_j (for $j = 0, \ldots, 4$) between the challenger and adversary $\mathcal{A}$, as illustrated in Figure 7. Let W_j be the event that $\mathcal{A}$'s output $\hat{b}$ equals the challenger's random bit b in game G_j.

Games G_0 to G_4:

$(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{Gen}_1(1^\lambda)$, $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$
$\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$, $\mathsf{sk} := (\mathsf{sk}_1, \mathsf{sk}_2)$
$(k_1^*, c_1^*) \leftarrow \mathsf{Encaps}_1(\mathsf{pk}_1)$, $(k_2^*, c_2^*) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$
$k_1^* \leftarrow\!\$\ \mathcal{K}_{\mathsf{kem}_1}$ // G_4
$k^* := (k_1^*, k_2^*)$, $c^* := (c_1^*, c_2^*)$
initialize an empty associative array
$Map : \mathcal{K}_{\mathsf{kem}_1} \times \mathcal{K}_{\mathsf{kem}_2} \to \mathrm{Range}(G)$
$k_0, k_1 \leftarrow\!\$\ \mathrm{Range}(G)$, $\bar{k} \leftarrow\!\$\ \mathrm{Range}(G)$
$Map[k^*] := k_0$ // $G_0 - G_1$
$b \leftarrow\!\$\ \{0,1\}$, $\mathsf{cnt} := 0$
$\mathsf{flag} \leftarrow\!\$\ \{0,1,2\}$, $i^* \leftarrow\!\$\ \{0, \cdots, q_G\}$ // $G_3 - G_4$
$\hat{b} \leftarrow \mathcal{A}^{G, O_{\mathsf{Dec}}}(\mathsf{pk}, c^*, k_b)$
return $[b = \hat{b}]$

Random Oracle $G(k)$:

if $\mathsf{flag} = 2$ **and** k is the i^*-th new query **then**
  $Map[k] := \bar{k}$ // $G_3 - G_4$
if $k \notin \mathrm{Domain}(Map)$ **then**
  $Map[k] \leftarrow\!\$\ \mathrm{Range}(G)$
return $Map[k]$

Decapsulation Oracle $O_{\mathsf{Dec}}(c \neq c^*)$:

if $\mathsf{cnt} \neq 0$ **then** : **return** $\perp$
$\mathsf{cnt} := \mathsf{cnt} + 1$
$(c_1, c_2) := c$
$k_1' := \mathsf{Decaps}_1(\mathsf{sk}_1, c_1)$
$k_2' := \mathsf{Decaps}_2(\mathsf{sk}_2, c_2)$
$k' := (k_1', k_2')$ // $G_0 - G_2$
if $k' = k^*$ **then** // $G_1 - G_2$
if $\mathsf{flag} = 0$ **then** // $G_3 - G_4$
  return `test` // $G_1 - G_4$
if $\perp \in \{k_1', k_2'\}$ **then** // $G_0 - G_2$
if $\mathsf{flag} = 1$ **then** // $G_3 - G_4$
  return $\perp$
$k := G(k')$ // $G_0 - G_2$
$k := \bar{k}$ // $G_3 - G_4$
if $k \in \{k_0, k_1\}$ **then**
  return `test`
return k

Figure 7. Games G_0 to G_4 for the proof of Theorem 3.

Game G_0. This game is defined as in Figure 7. In this game, the challenger initializes an empty associative array $Map : \mathcal{K}_{\text{kem}_1} \times \mathcal{K}_{\text{kem}_2} \to \text{Range}(G)$ to implement the random oracle G. Note that although the challenger initially selects a random $k_0 \leftarrow\!\!\$\ \text{Range}(G)$, it immediately sets $Map[k^*] := k_0$, which is equivalent to defining $G(k^*) = k_0$. Furthermore, although the challenger samples a random $\bar{k} \leftarrow\!\!\$\ \text{Range}(G)$, this value is not used in the subsequent game. Therefore, from the perspective of $\mathcal{A}$, the behavior of the challenger in G_0 is consistent with that in the IND-1RCCA game for $\mathsf{KEM}_{int.}$. Thus, we have

$$\mathsf{Adv}^{\text{IND-1RCCA}}_{\mathsf{KEM}_{int.}}(\mathcal{A}) = |\Pr[W_0] - 1/2| \,. \tag{1}$$

Game G_1. In this game, the decapsulation oracle O_{Dec} explicitly handles the case where $k' = k^*$ during decapsulation (as shown in lines 20 and 22 of Figure 7): when $k' = k^*$, O_{Dec} directly returns `test`. From the perspective of $\mathcal{A}$, this modification does not alter the return value of the decapsulation oracle. Hence,

$$\Pr[W_1] = \Pr[W_0] \,. \tag{2}$$

Game G_2. In this game, the challenger no longer sets $Map[k^*] := k_0$, i.e., line 8 in Figure 7 is removed. Let Z be the event that $\mathcal{A}$ queries the random oracle G on k^*. Note that if event Z does not occur, then $G(k^*)$ is independent and random from the view of $\mathcal{A}$, and thus $\Pr[W_2 \mid \bar{Z}] = \Pr[W_1 \mid \bar{Z}]$. By the Difference Lemma (Lemma 4), we have

$$|\Pr[W_2] - \Pr[W_1]| \leq \Pr[Z]. \tag{3}$$

Let Z_j be the event that $\mathcal{A}$ queries the random oracle G on $(k_1^*, \cdot)$ in game G_j. Clearly,

$$\Pr[Z] \leq \Pr[Z_2]. \tag{4}$$

Since both k_0 and k_1 are now independent and random from the view of $\mathcal{A}$, we have

$$\Pr[W_2] = 1/2. \tag{5}$$

Game G_3. In this game, the decapsulation oracle O_{Dec} no longer relies on the specific value of k' to compute the return value, but instead determines the output of O_{Dec} through a "guessing" strategy. At the beginning of the game, the challenger randomly selects a $\mathsf{flag} \leftarrow\!\!\$\ \{0, 1, 2\}$ to guess the possible scenarios for k' in the decapsulation oracle: $\mathsf{flag} = 0$ indicates that the challenger assumes $k' = k^*$, in which case O_{Dec} directly returns `test` (as shown in lines 21 and 22 of Figure 7); $\mathsf{flag} = 1$ indicates that the challenger assumes $\perp \in \{k_1', k_2'\}$ in the decapsulation oracle simulation, in which case O_{Dec} directly returns $\perp$ (as shown in lines 24 and 25 of Figure 7); and $\mathsf{flag} = 2$ corresponds to all other cases, where the challenger directly sets $k := \bar{k}$ (as shown in line 27 of Figure 7), and then determines the return value of O_{Dec} based on k. Here, $\bar{k} \leftarrow\!\!\$\ \text{Range}(G)$ is a random value pre-selected by the challenger at the start of the game. In the case of $\mathsf{flag} = 2$, the challenger further modifies the random oracle G to ensure consistency between the simulation of the decapsulation oracle and the random oracle: if $\mathsf{flag} = 2$ and k is the i^*-th new query, then $Map[k] := \bar{k}$ (see line 13 of Figure 7), where $i^* \leftarrow\!\!\$\ \{0, \ldots, q_G\}$ and q_G is the bounded number of G-queries made by $\mathcal{A}$.

Let E be the event that the challenger guesses correctly. We can see that:

- If $k' = k^*$, the decapsulation oracle O_{Dec} in both games G_2 and G_3 returns `test`, hence $\Pr[Z_2 \wedge k' = k^*] = \Pr[Z_3 \wedge k' = k^* \mid E]$.
- If $\perp \in \{k_1', k_2'\}$, the decapsulation oracle O_{Dec} in both games G_2 and G_3 returns $\perp$, hence $\Pr[Z_2 \wedge \perp \in \{k_1', k_2'\}] = \Pr[Z_3 \wedge \perp \in \{k_1', k_2'\} \mid E]$.

- If $k' \neq k^*$ and $\perp \notin \{k'_1, k'_2\}$, game G_2 sets $k := G(k')$, while game G_3 sets $k := \bar{k}$. Note that in this case, the challenger in G_3 further randomly guesses that the adversary $\mathcal{A}$ might query k' to the random oracle G at the i^*-th new query, and sets $Map[k] := \bar{k}$ ($i^* = 0$ means $\mathcal{A}$ does not query k' to the random oracle G). If the challenger guesses correctly, then O_{Dec} in G_3 effectively returns $G(k')$, making its behavior consistent with that of O_{Dec} in G_2. Since the probability that the challenger correctly guesses the index when $\mathcal{A}$ queries G on k' is $\Pr[\text{guess } i^*] = 1/(q_G + 1)$, we have

$$\begin{aligned}\Pr[Z_2 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}] &= \Pr[Z_3 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\} \mid E \wedge \text{guess } i^*] \\ &= \frac{\Pr[Z_3 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\} \wedge \text{guess } i^* \mid E]}{\Pr[\text{guess } i^*]} \\ &\leq (q_G + 1) \Pr[Z_3 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\} \mid E] \,.\end{aligned}$$

Therefore,

$$\begin{aligned}\Pr[Z_3 \mid E] &\geq \Pr[Z_2 \wedge k' = k^*] + \Pr[Z_2 \wedge \perp \in \{k'_1, k'_2\}] + \frac{\Pr[Z_2 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}]}{q_G + 1} \\ &\geq \Pr[Z_2]/q_G + 1 \,.\end{aligned}$$

Since $\Pr[E] = 1/3$, we have

$$\Pr[Z_3] \geq \Pr[Z_3 \wedge E] = \Pr[Z_3 \mid E] \cdot \Pr[E] \geq \Pr[Z_2]/3(q_G + 1) \,. \tag{6}$$

Game G_4. In this game, the challenger samples $k_1^* \leftarrow\!\!\$\ \mathcal{K}_{\mathrm{kem}_1}$ uniformly at random (line 4 of Figure 7) to compute the challenge key k^*, instead of using $(k_1^*, c_1^*) \leftarrow \mathsf{Encaps}_1(\mathsf{pk}_1)$. We now show that if $\mathcal{A}$ can distinguish between G_3 and G_4, we can construct an adversary $\mathcal{B}$ against the IND-CPA security of KEM_1.

Specifically, upon receiving $(\mathsf{pk}_1, k_{b'}, c_1^*)$ from the IND-CPA challenger of KEM_1, where b' is a random bit chosen by the challenger, $\mathcal{B}$ sets $k_1^* := k_{b'}$, computes $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$ and $(k_2^*, c_2^*) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$, and sets $\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$, $k^* := (k_1^*, k_2^*)$, and $c^* := (c_1^*, c_2^*)$. It then samples $k \leftarrow\!\!\$\ \mathrm{Range}(G)$ and sends (pk, c^*, k) to $\mathcal{A}$. The random oracle G and the decapsulation oracle O_{Dec} are simulated by $\mathcal{B}$ consistently with their behavior in G_3. At the end of the game, if $k_{b'} \in \mathrm{Domain}(Map)$, $\mathcal{B}$ outputs $\hat{b}' = 1$; otherwise, it outputs $\hat{b}' = 0$.

Note that when $b' = 0$, $k_{b'}$ is the real key, and $\mathcal{B}$'s behavior is identical to that of the challenger in G_3, so $\Pr[\hat{b}' = 1 \mid b' = 0] = \Pr[Z_3]$; when $b' = 1$, $k_{b'}$ is a random key, and $\mathcal{B}$'s behavior is identical to that of the challenger in G_4, so $\Pr[\hat{b}' = 1 \mid b' = 1] = \Pr[Z_4]$. Therefore, we have

$$\begin{aligned}\mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) &\geq \left|\Pr[b' = \hat{b}'] - 1/2\right| \\ &= \left|\Pr[\hat{b}' = 0 \wedge b' = 0] + \Pr[\hat{b}' = 1 \wedge b' = 1] - 1/2\right| \\ &= \left|\Pr[\hat{b}' = 0 \mid b' = 0]/2 + \Pr[\hat{b}' = 1 \mid b' = 1]/2 - 1/2\right| \\ &= \left|(1 - \Pr[\hat{b}' = 1 \mid b' = 0])/2 + \Pr[\hat{b}' = 1 \mid b' = 1]/2 - 1/2\right| \\ &= |\Pr[Z_4] - \Pr[Z_3]|/2 \,.\end{aligned} \tag{7}$$

Since in G_4, k_1^* is independent and random from the view of $\mathcal{A}$, we have

$$\Pr[Z_4] = q_G / \left|\mathcal{K}_{\mathrm{kem}_1}\right| \,. \tag{8}$$

Combining Equations (1)–(8), we obtain

$$\mathsf{Adv}^{\text{IND-1RCCA}}_{\mathsf{KEM}_{int.}}(\mathcal{A}) \leq 6(q_G+1)\cdot \mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) + 3q_G(q_G+1)/\left|\mathcal{K}_{\mathrm{kem}_1}\right| .$$

□

3.2.2. Security of the *Intermediate* KEM in the QROM

Theorem 4 (IND-1RCCA security of the hybrid $\mathsf{KEM}_{int.}$ in the QROM)**.** *Let* $\mathsf{KEM}_{int.} := \mathsf{T}_{int.}[\mathsf{Comb}[\mathsf{KEM}_1, \mathsf{KEM}_2], G]$ *(*Comb *and* $\mathsf{T}_{int.}$ *are defined in Figures 2 and 4) with G modeled as a quantum-accessible random oracle. If* KEM_1 *is* IND-CPA *secure, then* $\mathsf{KEM}_{int.}$ *is* IND-1RCCA *secure. Specifically, any efficient adversary* $\mathcal{A}$ *against* $\mathsf{KEM}_{int.}$ *implies an efficient adversary* $\mathcal{B}$ *against* KEM_1 *with*

$$\mathsf{Adv}^{\text{IND-1RCCA}}_{\mathsf{KEM}_{int.}}(\mathcal{A}) \leq 10(q_G+1)\sqrt{2\mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) + (q_G+1)^2/\left|\mathcal{K}_{\mathrm{kem}_1}\right| + 1/|G|} ,$$

where q_G *bounds* $\mathcal{A}$*'s queries to G,* $\mathcal{K}_{\mathsf{KEM}_1}$ *is the key space of* KEM_1*, and* $|G|$ *is the output space size of G.*

Proof of Theorem 4. We define a sequence of games G_j (for $j = 0, \ldots, 3$) between the challenger and adversary $\mathcal{A}$, as depicted in Figure 8. Let W_j denote the event that $\mathcal{A}$'s output $\hat{b}$ equals the challenger's random bit b in game G_j.

Games G_0 to G_3:

1 : $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{Gen}_1(1^\lambda)$, $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$
2 : $\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$, $\mathsf{sk} := (\mathsf{sk}_1, \mathsf{sk}_2)$
3 : $(k_1^*, c_1^*) \leftarrow \mathsf{Encaps}_1(\mathsf{pk}_1)$, $(k_2^*, c_2^*) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$
4 : $k^* := (k_1^*, k_2^*)$, $c^* := (c_1^*, c_2^*)$
5 : $\mathsf{G} \leftarrow\!\$\ \Omega_G$, $k_0, k_1 \leftarrow\!\$\ \mathrm{Range}(G)$
6 : $k_0 := G(k^*)$ // $G_0 - G_1$
7 : $b \leftarrow\!\$\ \{0,1\}$, $\mathsf{cnt} := 0$
8 : $\hat{b} \leftarrow\!\$\ \mathcal{A}^{|G\rangle, O_{\mathsf{Dec}}}(\mathsf{pk}, c^*, k_b)$
9 : **return** $[b = \hat{b}]$

Random Oracle $G(k)$:

10 : **if** $k = k^*$ **then** : **return** k_0 // G_3
11 : **return** $\mathsf{G}(k)$

Decapsulation Oracle $O_{\mathsf{Dec}}(c \neq c^*)$:

12 : **if** $\mathsf{cnt} \neq 0$ **then** : **return** $\perp$
13 : $\mathsf{cnt} := \mathsf{cnt} + 1$, $(c_1, c_2) := c$
14 : $k_1' := \mathsf{Decaps}_1(\mathsf{sk}_1, c_1)$
$k_2' := \mathsf{Decaps}_2(\mathsf{sk}_2, c_2)$
$k' := (k_1', k_2')$
15 : **if** $k' = k^*$ **then**
return `test` // $G_1 - G_3$
16 : **if** $\perp \in \{k_1', k_2'\}$ **then**
return $\perp$
17 : $k := G(k')$
18 : **if** $k \in \{k_0, k_1\}$ **then**
return `test`
19 : **return** k

Figure 8. Games G_0 to G_3 for the proof of Theorem 4.

Game G_0. This game is defined as in Figure 8. In this game, the challenger implements the random oracle G by sampling a random function $\mathsf{G} \leftarrow\!\$\ \Omega_G$. Note that although the challenger initially selects a random $k_0 \leftarrow\!\$\ \mathrm{Range}(G)$ (as shown in line 5 of Figure 8), it is immediately overwritten by setting $k_0 := G(k^*)$ (i.e., line 6). Therefore, from the perspective of $\mathcal{A}$, the behavior of the challenger in G_0 is identical to that in the IND-1RCCA game for $\mathsf{KEM}_{int.}$. Hence, we have

$$\mathsf{Adv}^{\text{IND-1RCCA}}_{\mathsf{KEM}_{int.}}(\mathcal{A}) = \left|\Pr[W_0] - 1/2\right| . \tag{9}$$

Game G_1. In this game, the decapsulation oracle O_{Dec} explicitly handles the case where $k' = k^*$ (see line 15 of Figure 8): if $k' = k^*$, it directly returns `test`. Note that this operation does not alter the behavior of O_{Dec}. Therefore,

$$\Pr[W_1] = \Pr[W_0]\,. \tag{10}$$

Game G_2. In this game, the challenger no longer resets $k_0 := G(k^*)$, i.e., line 6 in Figure 8 is removed. Since both k_0 and k_1 are now independent from the view of $\mathcal{A}$, we have

$$\Pr[W_2] = 1/2\,. \tag{11}$$

Game G_3. In this game, the challenger adds an additional operation to the simulation of the random oracle G: if the query $k = k^*$, it returns k_0. Note that this is equivalent to setting $G(k^*) := k_0$. From the perspective of $\mathcal{A}$, the behavior of the challenger is now consistent with that in G_1. Hence,

$$\Pr[W_3] = \Pr[W_1]\,. \tag{12}$$

Observe that compared to G_2, the challenger can be viewed as implementing the random oracle G using a new random function $\mathsf{G}' \in \Omega_G$, where $\mathsf{G}'(k) = \mathsf{G}(k)$ for all $k \neq k^*$ and $\mathsf{G}'(k^*) = k_0$. To bound $|\Pr[W_3] - \Pr[W_2]|$ using the DS-O2H Lemma (Lemma 1), we first construct two oracle algorithms $A^{|\mathsf{G}\rangle, O_{\mathsf{Dec}}}(z)$ and $A^{|\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)$ that simulate games G_2 and G_3, respectively. Here, $z := (\mathsf{pk}, c^*, k_0, k_1, b)$, and $(\mathsf{pk}, c^*, k_0, k_1, b)$ is generated consistently with game G_2: first compute $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{Gen}_1(1^\lambda)$ and $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$, then set $\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$; next compute $(k_1^*, c_1^*) \leftarrow \mathsf{Encaps}_1(\mathsf{pk}_1)$ and $(k_2^*, c_2^*) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$, and set $c^* := (c_1^*, c_2^*)$; finally, sample $k_0, k_1 \leftarrow\!\$\, \mathrm{Range}(G)$ and $b \leftarrow\!\$\, \{0,1\}$ uniformly at random. Let $\mathsf{G} \leftarrow\!\$\, \Omega_G$, and define $\mathsf{G}' \in \Omega_G$ such that $\mathsf{G}'(k) = \mathsf{G}(k)$ for all $k \neq k^*$ and $\mathsf{G}'(k^*) = k_0$, where $k^* := (k_1^*, k_2^*)$. The algorithm $A^{|\mathsf{G}\rangle, O_{\mathsf{Dec}}}(z)$ (resp. $A^{|\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)$) simulates the random oracle G as described in game G_2 (resp. G_3), runs $\hat{b} \leftarrow \mathcal{A}^{|G\rangle, O_{\mathsf{Dec}}}(\mathsf{pk}, c^*, k_b)$, and outputs $[b = \hat{b}]$. It is not hard to see that

$$\Pr[1 \leftarrow A^{|\mathsf{G}\rangle, O_{\mathsf{Dec}}}(z)] = \Pr[W_2] \quad \text{and} \quad \Pr[1 \leftarrow A^{|\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)] = \Pr[W_3]\,. \tag{13}$$

Since G and G' differ only at k^*, by the DS-O2H Lemma (Lemma 1), there exists an oracle algorithm $B^{|\mathsf{G}\rangle, |\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)$ such that

$$\left|\Pr[1 \leftarrow A^{|\mathsf{G}\rangle, O_{\mathsf{Dec}}}(z)] - \Pr[1 \leftarrow A^{|\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)]\right| \leq 2\sqrt{\Pr[k^* = \hat{k} : \hat{k} \leftarrow B^{|\mathsf{G}\rangle, |\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)]}\,, \tag{14}$$

where B makes at most q_G queries to $|\mathsf{G}\rangle$ and q_G is the upper bound on the number of G-queries made by $\mathcal{A}$. Note that in the simulation of O_{Dec} in G_2 and G_3, the query k' to the random oracle G (i.e., line 17 of Figure 8) is not equal to k^* (since the case $k' = k^*$ has been handled at line 15 by directly returning `test`). Therefore, the simulation of O_{Dec} is consistent in G_2 and G_3, and is independent of whether G is simulated using G or G'. Thus, when applying the DS-O2H Lemma, O_{Dec} can be treated as an internal oracle of A.

Next, we define a sequence of games G_j (for $j = 4, \ldots, 6$) played between a challenger and the oracle algorithm B, as shown in Figure 9. Here, k_1^* is the key computed by the challenger for KEM_1, and $\hat{k} = (\hat{k}_1, \hat{k}_2)$ is the key output by B. Let Z_j denote the event that $k_1^* = \hat{k}_1$ in game G_j.

Game G_4. In this game, the challenger samples a random function $\mathsf{G} \leftarrow\!\$\, \Omega_G$ to simulate the random oracles G and G': $G(k)$ directly returns $\mathsf{G}(k)$; $G'(k^*)$ returns k_0, while $G'(k \neq k^*)$ returns $G(k)$ (i.e., $\mathsf{G}(k)$). This is equivalent to B having direct access to G and G'. Note that although the challenger samples additional values in line 8, they are never actually used in the game. Therefore, we have

$$\Pr[k^* = \hat{k} : \hat{k} \leftarrow B^{|\mathsf{G}\rangle, |\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)] \leq \Pr[k_1^* = \hat{k}_1 : (\hat{k}_1, \hat{k}_2) := \hat{k} \leftarrow B^{|\mathsf{G}\rangle, |\mathsf{G}'\rangle, O_{\mathsf{Dec}}}(z)] = \Pr[Z_4]\,. \tag{15}$$

Games G_4 to G_6:

1 : $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{Gen}_1(1^\lambda)$, $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$
2 : $\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$, $\mathsf{sk} := (\mathsf{sk}_1, \mathsf{sk}_2)$
3 : $(k_1^*, c_1^*) \leftarrow \mathsf{Encaps}_1(\mathsf{pk}_1)$, $(k_2^*, c_2^*) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$
4 : $k_1^* \leftarrow\!\$\ \mathcal{K}_{\mathrm{kem}_1}$ // G_6
5 : $k^* := (k_1^*, k_2^*)$, $c^* := (c_1^*, c_2^*)$
6 : $\mathsf{G} \leftarrow\!\$\ \Omega_G$, $k_0, k_1 \leftarrow\!\$\ \mathrm{Range}(G)$
7 : $b \leftarrow\!\$\ \{0,1\}$, $\mathsf{cnt} := 0$
8 : $l := 0$, $\mathsf{flag} \leftarrow\!\$\ \{0,1,2\}$, $\bar{k} \leftarrow\!\$\ \mathrm{Range}(G)$
$(i,b) \leftarrow\!\$\ (\{1,\ldots,q_G\} \times \{0,1\}) \cup \{(q_G+1, 0)\}$
9 : $(\hat{k}_1, \hat{k}_2) := \hat{k} \leftarrow B^{|G\rangle, |G'\rangle, O_{\mathsf{Dec}}}(\mathsf{pk}, c^*, k_0, k_1, b)$
10 : **return** $[k_1^* = \hat{k}_1]$

Random Oracle $G(k)$:

11 : **if** $\mathsf{flag} = 2$ **then**
if $l \geq (i+b)$ and $k = k_i$ **then**
// k_i is the measurement outcome on
// B's i-th G-qeury intput register
return $\bar{k}$
$l = l + 1$ // $G_5 - G_6$
12 : **return** $\mathsf{G}(k)$

Decapsulation Oracle $O_{\mathsf{Dec}}(c \neq c^*)$:

13 : **if** $\mathsf{cnt} \neq 0$ **then** : **return** $\perp$
14 : $\mathsf{cnt} := \mathsf{cnt} + 1$, $(c_1, c_2) := c$
15 : $k_1' := \mathsf{Decaps}_1(\mathsf{sk}_1, c_1)$
$k_2' := \mathsf{Decaps}_2(\mathsf{sk}_2, c_2)$
$k' := (k_1', k_2')$ // G_4
16 : **if** $k' = k^*$ **then** // G_4
17 : **if** $\mathsf{flag} = 0$ **then** // $G_5 - G_6$
18 : **return** `test`
19 : **if** $\perp \in \{k_1', k_2'\}$ **then** // G_4
20 : **if** $\mathsf{flag} = 1$ **then** // $G_5 - G_6$
21 : **return** $\perp$
22 : $k := G(k')$ // G_4
23 : $k := \bar{k}$ // $G_5 - G_6$
24 : **if** $k \in \{k_0, k_1\}$ **then**
return `test`
25 : **return** k

Random Oracle $G'(k)$:

26 : **if** $k = k^*$ **then** : **return** k_0
27 : **return** $G(k)$

Figure 9. Games G_4 to G_6 for the proof of Theorem 4.

Game G_5. In this game, the challenger employs a "guessing" strategy to simulate the decapsulation oracle O_{Dec}, thereby eliminating its dependency on sk. The challenger first selects a random $\mathsf{flag} \leftarrow\!\$\ \{0,1,2\}$, and for the $k' = (k_1', k_2')$ in O_{Dec}: when $\mathsf{flag} = 0$, the challenger guesses $k' = k^*$ and directly outputs `test`; when $\mathsf{flag} = 1$, the challenger guesses $\perp \in \{k_1', k_2'\}$ and directly outputs $\perp$; when $\mathsf{flag} = 2$, the challenger guesses that k' falls into other cases, i.e., $k' \neq k^*$ and $\perp \notin \{k_1', k_2'\}$. In the last case, the challenger sets $k := \bar{k}$, where $\bar{k} \leftarrow\!\$\ \mathrm{Range}(G)$ is a random value pre-sampled by the challenger. If $k \in \{k_0, k_1\}$, O_{Dec} returns $\perp$; otherwise, it returns k.

In the case of $\mathsf{flag} = 2$, the challenger further modifies the random oracle G as follows: for the l-th G-query where $l \geq (i+b)$, G uses $\bar{k}$ as the return value for the query $k = k_i$. Here, $(i,b) \leftarrow\!\$\ (\{1,\ldots,q_G\} \times \{0,1\}) \cup \{(q_G+1, 0)\}$ is a random value pre-sampled by the challenger, and k_i is the measurement outcome of the challenger's input register for B's i-th G-query.

Let E be the event that the challenger guesses the correct flag. We can observe that:

- If $k' = k^*$, the decapsulation oracle O_{Dec} in both games G_4 and G_5 returns `test`, hence $\Pr[Z_4 \wedge k' = k^*] = \Pr[Z_5 \wedge k' = k^* \mid E]$.
- If $\perp \in \{k_1', k_2'\}$, the decapsulation oracle O_{Dec} in both games G_4 and G_5 returns $\perp$, hence $\Pr[Z_4 \wedge \perp \in \{k_1', k_2'\}] = \Pr[Z_5 \wedge \perp \in \{k_1', k_2'\} \mid E]$.
- If $k' \neq k^*$ and $\perp \notin \{k_1', k_2'\}$, we can use Lemma 3 to derive that $\Pr[Z_4 \wedge k' \neq k^* \wedge \perp \notin \{k_1', k_2'\}] \leq 8(q_G+1)^2(\Pr[Z_5 \wedge k' \neq k^* \wedge \perp \notin \{k_1', k_2'\} \mid E] + 1/|G|)$ as follows, where $|G|$ is the output space size of G.

Conditioned on $k' \neq k^*$ and $\perp \notin \{k_1', k_2'\}$, we can construct an oracle algorithm $C^{|G\rangle}$ that generates $(\mathsf{pk}, \mathsf{sk}, k^*, c^*, k_0, k_1, b)$ and runs $B^{|G\rangle, |G'\rangle, O_{\mathsf{Dec}}}$ as the challenger does in G_4. Let $\bar{c} = (\bar{c}_1, \bar{c}_2)$ be the decapsulation oracle query made by B, and define $k_1' := \mathsf{Decaps}_1(\mathsf{sk}_1, \bar{c}_1)$, $k_2' := \mathsf{Decaps}_2(\mathsf{sk}_2, \bar{c}_2)$, $k' := (k_1', k_2')$. Define $x := k'$, $y := G(x)$, and $z := (z_1, z_2, z_3) = (k, k_1^*, \hat{k}_1)$, where k is the value used in O_{Dec} to determine its final output (i.e., the k in line 24

of Figure 9). If k is undefined, i.e., when $k' = k^*$ or $\perp \in \{k'_1, k'_2\}$, we set $k := \perp$. Define the predicates $V_1(x, y, z) := [y = z_1]$ and $V_2 := [z_2 = z_3]$. Let $V := V_1 \wedge V_2$. Note that in game G_4, when $k' \neq k^*$ and $\perp \notin \{k'_1, k'_2\}$, $V_1(x, y, z)$ always holds. Therefore, we can see that

$$\begin{aligned}\Pr[Z_4 \mid k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}] &= \Pr_G[V(x, y, z) = 1 : (x, z) \leftarrow C^{|G\rangle}] \\ &= \sum_{x_0 \in \mathcal{K}_{\mathrm{kem}_1} \times \mathcal{K}_{\mathrm{kem}_2}} \Pr_G \Pr[x = x_0 \wedge V(x, y, z) = 1 : (x, z) \leftarrow C^{|G\rangle}] .\end{aligned}$$

It is important to note that in game G_4, besides the q_G oracle G queries made by the oracle algorithm B, the decapsulation oracle O_{Dec} implicitly makes at most one additional (classical) query to $G(k')$ (i.e., line 22 in Figure 9). Thus, C makes a total of $q_G + 1$ random oracle G queries. Unless otherwise specified, the G queries mentioned below exclude the implicit G query in O_{Dec}. Let $S^C(\Theta)$ be the oracle algorithm that always uses Θ to respond to C's implicit G query $G(k')$. This algorithm randomly samples $(i, b) \leftarrow\!\$\ (\{1, \ldots, q_G\} \times \{0, 1\}) \cup \{(q_G + 1, 0)\}$, runs $C^{|G\rangle}$ until just before C's i-th G query (excluding the implicit query), measures C's input register to obtain x, continues running $C^{|G\rangle}$ until its $(i + b)$-th G query, reprograms G to $G_{x\Theta}$ (where $G_{x\Theta}(x') = G(x')$ for all $x' \neq x$ and $G_{x\Theta}(x) = \Theta$), and continues running $C^{|G_{x\Theta}\rangle}$ until it outputs z. Let $y := \Theta$ and $z := (z_1, z_2, z_3) = (k, k_1^*, \hat{k}_1)$, where k is the value used in O_{Dec} to determine its final output (i.e., the k in line 24 of Figure 9). Note that $V_1(x, y, z) = [y = z_1]$ still holds here. Therefore, when $\Theta \leftarrow\!\$\ \mathrm{Range}(G)$, under the condition $k' \neq k^*$ and $\perp \notin \{k'_1, k'_2\}$, $S^C(\Theta)$ perfectly simulates game G_5 when $\mathsf{flag} = 2$. Hence,

$$\begin{aligned}\Pr[Z_5 \mid k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\} \wedge E] &= \Pr_{G,\Theta}[V(x, \Theta, z) = 1 : (x, z) \leftarrow S^C(\Theta)] \\ &= \sum_{x_0 \in \mathcal{K}_{\mathrm{kem}_1} \times \mathcal{K}_{\mathrm{kem}_2}} \Pr_{G,\Theta}[x = x_0 \wedge V(x, \Theta, z) = 1 : (x, z) \leftarrow S^C(\Theta)] .\end{aligned}$$

By Lemma 3, we have

$$\begin{aligned}&\sum_{x_0 \in \mathcal{K}_{\mathrm{kem}_1} \times \mathcal{K}_{\mathrm{kem}_2}} \Pr_G \Pr[x = x_0 \wedge V(x, y, z) = 1 : (x, z) \leftarrow C^{|G\rangle}] \\ &\leq 2(2q_G + 1)^2 \sum_{x_0 \in \mathcal{K}_{\mathrm{kem}_1} \times \mathcal{K}_{\mathrm{kem}_2}} \Pr_{G,\Theta}[x = x_0 \wedge V(x, \Theta, z) = 1 : (x, z) \leftarrow S^C(\Theta)] + \frac{8(q_G + 1)^2}{|G|},\end{aligned}$$

where $|G|$ is the size of the output space of G. Therefore, we have

$$\begin{aligned}&\Pr[Z_4 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}] = \Pr[Z_4 | k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}] \Pr[k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}] \\ &\leq \left(8(q_G + 1)^2 \left(\Pr[Z_5 \mid k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\} \wedge E] + \frac{1}{|G|}\right)\right) \Pr[k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}] \\ &\leq 8(q_G + 1)^2 \big(\Pr[Z_5 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\} \mid E] + 1/|G|\big).\end{aligned}$$

Consequently, we have

$$\begin{aligned}\Pr[W_5] &\geq \big(\Pr[Z_5 \wedge k' = k^* \mid E] + \Pr[Z_5 \wedge \perp \in \{k'_1, k'_2\} \mid E] \\ &\qquad + \Pr[Z_5 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\} \mid E]\big) \Pr[E] \\ &\geq \big(\Pr[Z_4 \wedge k' = k^*] + \Pr[Z_4 \wedge \perp \in \{k'_1, k'_2\}]\big) \Pr[E] \\ &\qquad + \frac{\Pr[Z_4 \wedge k' \neq k^* \wedge \perp \notin \{k'_1, k'_2\}] \Pr[E]}{8(q_G + 1)^2} - \frac{1}{|G|} \\ &\geq \Pr[Z_4]/24(q_G + 1)^2 - 1/|G| .\end{aligned} \tag{16}$$

Game G_6. In this game, the challenger replaces k_1^* in $k^* := (k_1^*, k_2^*)$ with a random $k_1^* \leftarrow\!\$\ \mathcal{K}_{\mathrm{kem}_1}$ (as shown in line 4 of Figure 9). Note that in both games G_5 and G_6, the simulation of the decapsulation oracle O_{Dec} no longer uses sk. We proceed to construct a new adversary $\mathcal{B}$ against the IND-CPA security of KEM_1.

Specifically, upon receiving $(\mathsf{pk}_1, k_{b'}, c_1^*)$ from the IND-CPA challenger of KEM_1, where b' is a random bit chosen by the IND-CPA challenger of KEM_1, $\mathcal{B}$ sets $k_1^* := k_{b'}$, then computes $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{Gen}_2(1^\lambda)$ and $(k_2^*, c_2^*) \leftarrow \mathsf{Encaps}_2(\mathsf{pk}_2)$. It sets $\mathsf{pk} := (\mathsf{pk}_1, \mathsf{pk}_2)$, $k^* := (k_1^*, k_2^*)$, and $c^* := (c_1^*, c_2^*)$, samples $k_0, k_1 \leftarrow\!\$\ \mathrm{Range}(G)$ and $b \leftarrow\!\$\ \{0,1\}$ uniformly at random, and defines $z := (\mathsf{pk}, c^*, k_0, k_1, b)$. It then runs $B^{|G\rangle,|G'\rangle,O_{\mathsf{Dec}}}(z)$, where the random oracles G, G' and the decapsulation oracle O_{Dec} are simulated consistently with the challenger's behavior in G_5. When $B^{|G\rangle,|G'\rangle,O_{\mathsf{Dec}}}(z)$ outputs $\hat{k} = (\hat{k}_1, \hat{k}_2)$, $\mathcal{B}$ outputs $\hat{b}' := [k_1^* = \hat{k}_1]$.

Note that when $b' = 0$, $k_{b'}$ is the real key, and thus $\mathcal{B}$'s behavior is identical to that of the challenger in G_5, so $\Pr[\hat{b}' = 1 \mid b' = 0] = \Pr[Z_5]$; when $b' = 1$, $k_{b'}$ is a random key, and thus $\mathcal{B}$'s behavior is identical to that of the challenger in G_6, so $\Pr[\hat{b}' = 1 \mid b' = 1] = \Pr[Z_6]$. Hence, we have

$$\begin{aligned}
\mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) &\geq \left|\Pr[b' = \hat{b}'] - 1/2\right| \\
&= \left|\Pr[\hat{b}' = 0 \wedge b' = 0] + \Pr[\hat{b}' = 1 \wedge b' = 1] - 1/2\right| \\
&= \left|\Pr[\hat{b}' = 0 \mid b' = 0]/2 + \Pr[\hat{b}' = 1 \mid b' = 1]/2 - 1/2\right| \\
&= \left|(1 - \Pr[\hat{b}' = 1 \mid b' = 0])/2 + \Pr[\hat{b}' = 1 \mid b' = 1]/2 - 1/2\right| \\
&= |\Pr[Z_6] - \Pr[Z_5]|/2\,.
\end{aligned} \tag{17}$$

Since in game G_6, k_1^* is independent of the random oracle G, the decapsulation oracle O_{Dec}, and the input z to B, and B makes at most $q_G + 1$ random oracle G queries, by Lemma 2, we have

$$\Pr[Z_6] \leq (q_G + 1)^2 / \left|\mathcal{K}_{\mathrm{kem}_1}\right|\,. \tag{18}$$

Combining Equations (9)–(18), we obtain

$$\mathsf{Adv}^{\text{IND-1RCCA}}_{\mathsf{KEM}_{int.}}(\mathcal{A}) \leq 10(q_G + 1)\sqrt{2\mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{KEM}_1}(\mathcal{B}) + (q_G + 1)^2 / \left|\mathcal{K}_{\mathrm{kem}_1}\right| + 1/|G|}\,.$$

□

Remarks on Theorem 4. We should note that since random oracles are usually implemented using public hash functions in practice, a quantum adversary can query them in superposition "off-line", which is captured by the QROM. Therefore, a security reduction in the QROM, which bounds a quantum adversary's advantage against the obtained scheme by the advantage against a underlying primitive, ensures that the scheme's security holds against quantum adversaries, assuming the primitive is secure. This is precisely the approach taken in the proof of Theorem 4, where the adversary is given quantum access to oracle G, and the analysis from Game G_3 to G_5 addresses the implications of this access. Theorem 4 concludes that even in the presence of such a quantum adversary $\mathcal{A}$, $\mathsf{KEM}_{int.}$ is secure (i.e., $\mathsf{Adv}^{\text{IND-1RCCA}}_{\mathsf{KEM}_{int.}}(\mathcal{A})$ is negligible), as long as the underlying KEM_1 is secure (i.e., $\mathsf{Adv}^{\text{IND-CPA}}_{\mathsf{KEM}_1}(\mathcal{B})$ is negligible), and provided the key space size $\left|\mathcal{K}_{\mathrm{kem}_1}\right|$ and the output space size $|G|$ of the hash function G are sufficiently large.

3.2.3. Security of TLS 1.3 Hybrid Handshake from IND-CPA Secure Base KEM

Combining Theorems 3 and 4 with the results from [12], we finally establish the complete security of TLS 1.3 handshake with hybrid key exchange from IND-CPA secure KEMs.

Theorem 5 (Security of TLS 1.3 handshake with hybrid key exchange from IND-CPA secure KEM)**.** *If the other components in TLS 1.3 handshake are secure (as defined in [12]), then the entire TLS 1.3 with hybrid key exchange is secure, provided that at least one base KEM is* IND-CPA *secure.*

Proof of Theorem 5. This result follows by combining Theorem 3 with Theorem 4.1 of [12] (which establishes that IND-1RCCA security of the *intermediate* KEM implies the special IND-1CCA-1MAC security in the ROM), and Theorem 4 with Theorem 4.2 of the same reference (which proves the same implication in the QROM), and finally applying Theorems 6.2 and 6.3 of [12] (which show that IND-1CCA-1MAC security of the *intermediate* KEM implies multi-stage security for the TLS 1.3 handshake). □

4. Experimental Evaluation and Analysis

We have shown that CPA-secure KEMs suffice for the TLS 1.3 hybrid handshake. In contrast, current deployments (including IETF drafts [21,22]) rely on stronger IND-CCA secure KEMs like ML-KEM, which use the Fujisaki-Okamoto (FO) transform. This not only adds computational cost and complicates post-quantum security analysis but may also introduce side-channel risks. Adopting simpler IND-CPA secure KEMs (without the FO transform, as in Figure 1) could thus improve operational efficiency and reduce such risks. To ensure the reliability of our results, we performed experiments on both the x86-64 (Intel Core i7-1165G7 CPU) and ARM (Apple M4 chip, designed by Apple Inc., Cupertino, CA, USA; manufactured by TSMC, Hsinchu, Taiwan, China) architectures. We provide the complete source code and scripts for this work under the MIT License to ensure reproducibility and to support future research. Our public repository is https://github.com/CPA-TLS/CPA-TLS (accessed on 1 December 2025).

4.1. Performance Evaluation at the Hybrid Key Exchange Level

To quantify the performance improvement gained by replacing IND-CCA secure KEMs with IND-CPA secure ones, we modified the implementation of ML-KEM in the latest OpenSSL by removing the FO transform according to the method depicted in Figure 1, thereby obtaining its IND-CPA secure variant. We then used the `openssl speed` tool in a terminal to measure the computational cost of the three key exchange algorithms implemented in OpenSSL 3.6.0 (which correspond to those defined in the IETF draft [21]), using both the original IND-CCA secure ML-KEM implementation and our modified IND-CPA secure version. The experimental results are presented in Table 1, with a corresponding visualization shown in Figure 10. Each algorithm was executed for a default duration of 3 s, and the average time per operation was computed over 50 iterations. The raw data are available in Appendix A.

Table 1. Performance comparison of hybrid key exchange. Values are expressed as the mean ± the margin of error of the 95% confidence interval.

Scheme	Encaps (µs)		Decaps (µs)	
	CCA	CPA	CCA	CPA
X25519MLKEM768 †	134.82 ± 2.81	129.60 ± 1.25	104.16 ± 1.43	69.93 ± 0.79
SecP256r1MLKEM768 †	143.67 ± 1.45	140.98 ± 1.50	140.48 ± 1.41	105.14 ± 1.15
SecP384r1MLKEM1024 †	2523.1 ± 25.25	2380.7 ± 26.03	1317.1 ± 15.77	1197.6 ± 13.89
X25519MLKEM768 ‡	38.14 ± 0.39	37.80 ± 0.10	30.86 ± 0.13	23.45 ± 0.24
SecP256r1MLKEM768 ‡	44.65 ± 0.29	45.04 ± 0.31	42.53 ± 0.18	35.04 ± 0.08
SecP384r1MLKEM1024 ‡	753.96 ± 3.34	747.47 ± 2.47	392.21 ± 2.39	378.29 ± 2.36

† Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Intel Core i7-1165G7 CPU, 16 GB RAM.
‡ Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Apple M4 chip, 16 GB unified memory.

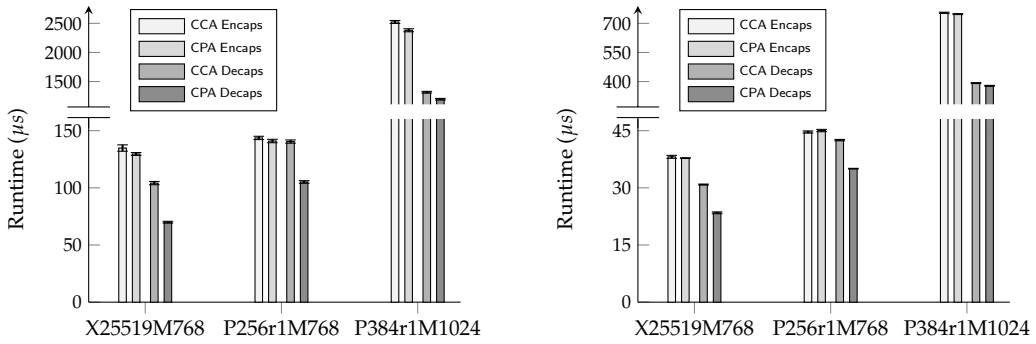


Figure 10. Performance comparison of hybrid key exchange. Bar heights show the mean, and error bars represent the 95% confidence interval. Labels "X25519M768", "P256r1M768", and "P384r1M1024" denote "X25519MLKEM768", "SecP256r1MLKEM768", and "SecP384r1MLKEM1024", respectively. (**a**) Experimental environment: Ubuntu 24.04.3 TLS VM, Intel Core i7-1165G5 CPU, 16 GB RAM. (**b**) Experimental environment: Ubuntu 24.04.3 TLS VM, Apple M4 chip, 16 GB unified memory.

From the results, we observe that the FO transform primarily impacts the performance of the decapsulation algorithm. Consequently, the execution time of the encapsulation algorithm shows no significant change before and after the modification. In contrast, the decapsulation algorithm exhibits performance improvements of approximately 44.8%, 25.2%, and 9.1% on the Intel Core i7-1165G7 CPU, and 24.0%, 17.6%, and 3.5% on the Apple M4 chip, for X25519MLKEM768, SecP256r1MLKEM768, and SecP384r1MLKEM1024, respectively, as expected. The most substantial improvement for X25519MLKEM768 can be attributed to the fact that within the hybrid key exchange, the X25519 (EC)DH component is faster than SecP256r1 and SecP384r1. Thus, the optimization achieved by removing the FO transform has a more pronounced effect on the overall performance of X25519MLKEM768.

4.2. Performance Evaluation at the Protocol Level

For performance testing at the full TLS 1.3 protocol level, we simulated a client and server locally using two separate terminals to measure protocol efficiency, thereby excluding external interference such as network latency. We first launched one terminal, generated a certificate for the server, and then used the command

```
openssl s_server -cert server.crt -key server.key -accept 4443 -www
-groups <SELECTED_GROUPS>
```

to start a server listening on port 4443. The `-groups` parameter specifies the key exchange algorithm for the handshake, i.e., `<SELECTED_GROUPS>` is set to one of X25519MLKEM768, SecP256r1MLKEM768, or SecP384r1MLKEM1024. Another terminal runs the command

```
openssl s_time -connect localhost:4443 -new -time 3
```

to initiate TLS 1.3 handshakes and run this for 3 s, and we measure the total connections established across 50 repetitions of the experiment. Given that the removal of the FO transform mainly impacts the decapsulation algorithms in the client, we focus on evaluating the average user time per TLS 1.3 handshake connection with different hybrid key exchange algorithms.

To include the aforementioned three hybrid key exchange algorithms in OpenSSL's default list, we modified the OpenSSL configuration file (`openssl.cnf`), specifying the supported groups in the `[tls_system_default]` section:

```
Groups = X25519MLKEM768:SecP256r1MLKEM768:SecP384r1MLKEM1024
```

Further details on this configuration can be found in the official OpenSSL documentation (https://openssl-library.org/post/2022-10-21-tls-groups-configuration/ accessed on 23 October 2025). The experimental results are presented in Table 2 and their corresponding visualization is in Figure 11. The raw data are available in Appendix B.

Table 2. Performance comparison of TLS 1.3 handshake with different hybrid key exchange schemes (user time/connection). Values are the mean ± the margin of error of the 95% confidence interval.

Scheme	CCA (µs)	CPA (µs)
X25519MLKEM768 †	721.81 ± 15.74	688.28 ± 13.04
SecP256r1MLKEM768 †	953.15 ± 19.64	907.63 ± 16.96
SecP384r1MLKEM1024 †	3134.9 ± 40.47	3012.8 ± 46.69
X25519MLKEM768 ‡	174.89 ± 1.38	163.50 ± 1.12
SecP256r1MLKEM768 ‡	229.46 ± 0.88	221.32 ± 0.61
SecP384r1MLKEM1024 ‡	957.00 ± 1.69	942.33 ± 1.54

† Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Intel Core i7-1165G7 CPU, 16 GB RAM.
‡ Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Apple M4 chip, 16 GB unified memory.

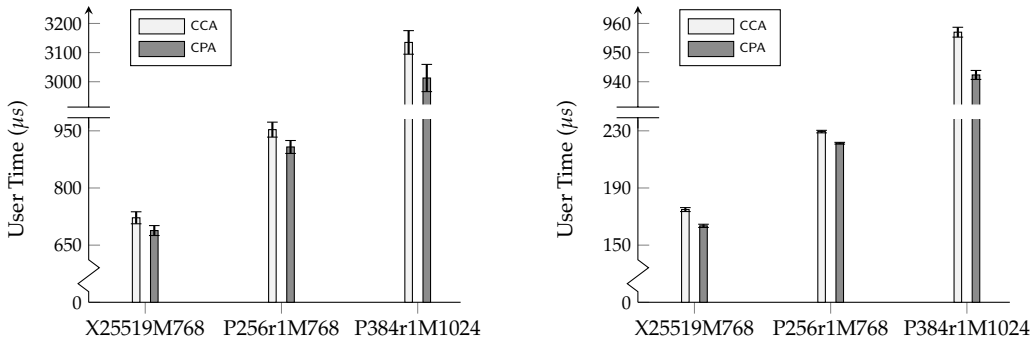


Figure 11. Performance comparison of TLS 1.3 handshake. Bar heights show the mean, and error bars represent the 95% confidence interval. Labels "X25519M768", "P256r1M768", and "P384r1M1024" denote "X25519MLKEM768", "SecP256r1MLKEM768", and "SecP384r1MLKEM1024", respectively. (**a**) Experimental environment: Ubuntu 24.04.3 TLS VM, Intel Core i7-1165G5 CPU, 16 GB RAM. (**b**) Experimental environment: Ubuntu 24.04.3 TLS VM, Apple M4 chip, 16 GB unified memory.

Compared to Table 1 and Figure 10, although removing the FO transform significantly improves the hybrid key exchange performance itself, the resulting efficiency gain at the overall protocol level is more modest, not exceeding 9% (considering the 95% confidence intervals). This is because the FO transform primarily impacts the runtime of KEM.Decaps, but during a single TLS connection, the client must perform one KEM.Gen, at least two digital signature verifications (for the certificate and the `CertificateVerify` signature), one MAC verification, and several key derivation operations, in addition to the KEM.Decaps. Consequently, while a positive performance gain is observed, employing a KEM with weaker security parameters yields a less significant efficiency improvement for the overall protocol than for the KEM operation itself.

Furthermore, it is important to note that removing the FO transform reduces the number of hash function invocations within the hybrid key exchange. As pointed out in Meta's post-quantum transition update presented at the "Real World PQC Workshop" [23], the "extra hashing steps" used in transforms like FO can lead to "non-negligible capacity cost" when deploying post-quantum key exchange, which "can cost hundreds of thousands or even millions of dollars a year". Scaling this to the global volume of TLS 1.3 traffic makes CPA-secure KEMs a more sustainable path for the post-quantum transition.

5. Conclusions

In this work, we have comprehensively demonstrated that CPA-secure KEMs are not only theoretically sufficient but also practically advantageous for the TLS 1.3 handshake in a hybrid key exchange setting. We bridged a critical gap between theoretical security models and practical deployment needs by providing the first formal security proofs, in both the ROM and QROM, that the security of the handshake can be reduced to the OW-CPA and IND-CPA security of a single component KEM within the hybrid construction.

Our security analysis is complemented by a rigorous performance evaluation. The experimental results substantiate a significant performance gain at the key exchange layer, with decapsulation speed improvements of up to 44.8%. While the overall protocol-level improvement is more modest, the elimination of the FO transform's re-encryption step consistently enhances efficiency and, more importantly, reduces the codebase's attack surface and potential operational costs.

This work firmly establishes CPA-secure KEMs as a viable, secure, and more efficient alternative for the ongoing standardization and deployment of post-quantum TLS 1.3. By relaxing the security requirement from IND-CCA to IND-CPA, we pave the way for simpler, faster, and potentially more secure implementations, thereby facilitating a smoother and more economical transition to the post-quantum era.

Author Contributions: Conceptualization, J.C.; methodology, J.C.; software, J.C. and Y.B.; validation, J.C.; formal analysis, J.C.; investigation, Y.B.; resources, Y.W.; data curation, J.C.; writing—original draft preparation, J.C.; writing—review and editing, J.C. and Y.W.; visualization, J.C.; supervision, W.P.; project administration, W.P.; funding acquisition, Y.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Natural Science Foundation of China, grant number 62202485.

Institutional Review Board Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in this article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Raw Data for the Performance Evaluation at the Hybrid Key Exchange Level

We employed two different processor architectures for the hybrid key exchange level experiments: an Intel Core i7-1165G7 CPU (x86-64) and an Apple M4 chip (ARM).

Appendix A.1. Raw Data on x86-64 Architecture at the Hybrid Key Exchange Level

The x86-64 benchmark configuration comprised an Ubuntu 24.04.3 LTS virtual machine equipped with an Intel Core i7-1165G7 CPU and 16 GB of RAM.

Table A1. Raw Data on x86-64 Architecture for the Performance Evaluation at the Hybrid Key Exchange Level for All 50 Runs (Time in μs).

Test Run	X25519MLKEM768				SecP256r1MLKEM768				SecP384r1MLKEM1024			
	Encaps		Decaps		Encaps		Decaps		Encaps		Decaps	
	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA
Run 1	132.97	129.22	102.64	67.90	138.24	148.47	135.21	100.82	2508.80	2300.54	1452.72	1125.38
Run 2	129.98	128.49	103.44	65.98	143.38	131.18	143.91	102.38	2740.74	2434.64	1360.29	1201.61
Run 3	128.03	125.72	101.13	67.75	141.09	138.59	139.58	107.39	2420.54	2343.26	1318.92	1144.83
Run 4	123.21	132.96	99.64	72.53	143.85	136.75	152.96	102.91	2651.79	2338.58	1303.20	1192.48
Run 5	130.71	127.15	105.55	71.08	142.74	136.82	146.79	104.95	2656.53	2338.58	1267.01	1225.83
Run 6	127.43	122.78	99.34	68.08	139.21	143.28	138.73	105.79	2462.81	2397.43	1275.23	1184.89

Table A1. *Cont.*

Test Run	X25519MLKEM768				SecP256r1MLKEM768				SecP384r1MLKEM1024			
	Encaps		Decaps		Encaps		Decaps		Encaps		Decaps	
	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA
Run 7	128.93	133.21	98.86	74.28	145.42	133.58	138.80	110.51	2571.18	2402.91	1220.12	1199.20
Run 8	121.85	134.09	98.47	69.61	143.40	139.66	135.75	102.19	2485.40	2468.93	1222.13	1227.35
Run 9	127.88	131.29	101.95	68.24	138.37	142.00	139.10	104.98	2305.32	2389.89	1295.25	1161.06
Run 10	132.95	121.41	102.41	62.55	146.58	141.48	138.75	98.07	2434.64	2458.75	1306.82	1195.65
Run 11	134.12	125.30	95.24	71.67	146.91	132.82	142.85	105.00	2429.98	2384.00	1282.38	1177.17
Run 12	127.55	130.12	102.69	68.12	142.10	146.72	141.82	101.01	2582.32	2365.08	1317.18	1162.25
Run 13	133.85	128.02	110.73	68.82	134.65	131.99	139.08	105.84	2466.78	2321.57	1308.17	1188.95
Run 14	133.19	122.61	103.18	70.95	143.28	138.97	140.29	103.62	2424.74	2448.30	1340.81	1169.29
Run 15	138.19	132.93	104.53	71.99	149.41	135.23	135.38	96.25	2531.75	2339.92	1328.58	1176.24
Run 16	127.71	125.89	100.59	68.79	143.48	142.25	142.16	107.27	2620.93	2232.21	1294.37	1247.90
Run 17	178.33	129.00	119.19	68.40	141.29	147.04	137.57	103.74	2500.00	2335.42	1230.39	1162.52
Run 18	165.86	123.60	97.68	71.09	137.44	141.32	136.42	97.74	2523.21	2355.73	1295.81	1127.51
Run 19	151.38	131.04	101.60	70.38	140.52	143.82	134.62	104.78	2627.80	2329.95	1300.35	1155.78
Run 20	127.54	133.47	104.21	65.78	141.34	137.85	145.96	106.21	2580.09	2321.57	1257.91	1251.57
Run 21	136.27	129.67	119.56	70.32	151.32	135.69	142.08	106.22	2434.43	2374.10	1289.91	1188.95
Run 22	144.48	136.41	103.54	71.40	137.64	147.75	136.70	104.86	2469.82	2244.74	1323.46	1266.47
Run 23	129.39	133.17	114.84	65.49	149.64	145.72	143.81	103.05	2542.52	2426.47	1298.47	1248.95
Run 24	132.91	125.38	105.82	76.37	146.44	132.35	143.59	101.04	2393.57	2315.46	1268.62	1157.57
Run 25	135.78	126.06	104.55	67.74	143.39	137.91	155.04	99.36	2508.42	2365.08	1316.94	1304.16
Run 26	136.55	123.66	101.94	65.70	140.51	142.30	139.80	95.80	2434.21	2306.50	1305.30	1210.53
Run 27	130.76	139.63	101.80	71.11	138.27	133.31	143.05	99.28	2516.95	2799.25	1199.84	1380.91
Run 28	133.34	129.17	106.33	66.60	154.04	130.81	129.99	104.00	2655.42	2284.62	1333.93	1231.85
Run 29	129.09	125.36	105.97	70.04	141.76	140.78	144.60	110.58	2609.46	2345.10	1333.93	1210.40
Run 30	139.33	131.29	103.23	67.68	140.05	136.65	147.17	106.78	2531.86	2355.73	1316.84	1211.38
Run 31	139.79	128.07	111.57	71.91	158.39	145.72	137.63	107.58	2580.09	2374.10	1231.40	1254.21
Run 32	154.05	134.08	101.95	73.12	145.73	139.26	140.31	108.70	2452.83	2616.33	1342.07	1224.83
Run 33	128.02	135.92	106.04	72.87	142.58	147.78	139.09	106.83	2475.17	2381.72	1423.78	1204.53
Run 34	128.28	132.77	102.46	73.57	139.88	142.35	142.99	111.38	2635.31	2374.50	1338.76	1163.42
Run 35	134.87	134.28	109.94	69.87	142.42	150.08	135.68	110.13	2722.27	2382.09	1371.38	1180.20
Run 36	134.76	131.79	103.51	71.71	151.86	149.82	138.32	112.38	2525.42	2381.72	1433.38	1142.20
Run 37	132.31	129.24	106.70	71.81	139.88	140.19	138.19	106.84	2523.36	2314.31	1325.35	1175.08
Run 38	140.11	129.81	107.56	68.64	137.58	139.78	131.59	105.44	2642.35	2416.87	1267.67	1226.26
Run 39	135.46	126.98	105.58	73.32	148.13	139.83	138.39	107.50	2477.06	2274.81	1447.43	1119.88
Run 40	132.62	127.79	105.98	74.99	144.80	150.79	143.56	105.03	2587.11	2456.72	1400.76	1228.36
Run 41	126.32	138.42	113.14	70.82	146.05	143.16	140.42	112.33	2479.20	2350.16	1324.44	1228.87
Run 42	136.61	135.77	102.24	67.93	140.24	141.69	136.70	105.42	2470.69	2428.69	1293.96	1150.58
Run 43	138.52	133.84	103.35	70.51	141.54	142.03	146.61	105.70	2557.94	2442.62	1318.00	1216.33
Run 44	127.88	121.98	102.24	66.02	147.63	139.32	136.29	111.23	2454.84	2462.69	1324.71	1132.29
Run 45	125.76	124.36	101.30	67.57	141.27	142.88	135.92	103.77	2469.03	2479.20	1396.54	1248.42
Run 46	136.16	130.88	98.56	68.45	143.36	146.90	140.63	103.61	2536.29	2283.52	1276.96	1213.85
Run 47	132.83	133.60	97.02	72.18	141.99	138.35	149.65	109.77	2575.89	2428.69	1356.16	1139.14
Run 48	134.26	126.67	101.31	72.91	159.25	141.11	143.69	111.63	2468.72	2315.46	1354.31	1201.29
Run 49	138.27	126.89	100.26	71.97	146.03	146.81	133.32	105.91	2510.57	2416.87	1343.01	1144.95
Run 50	134.34	128.77	106.51	69.93	139.27	148.06	143.43	103.57	2387.46	2297.61	1320.00	1197.27
Mean	134.82	129.60	104.16	69.93	143.67	140.98	140.48	105.14	2523.07	2380.69	1317.10	1197.61
SD	9.90	4.38	5.04	2.78	5.11	5.28	4.97	4.05	88.83	91.59	55.49	48.86
95%CI(LB)	132.00	128.35	102.73	69.14	142.22	139.48	139.07	103.99	2497.83	2354.66	1301.33	1183.73
95%CI(UB)	137.63	130.85	105.59	70.72	145.12	142.48	141.89	106.29	2548.32	2406.73	1332.87	1211.50

Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Intel Core i7-1165G7 CPU, 16 GB RAM.

Appendix A.2. Raw Data on ARM Architecture at the Hybrid Key Exchange Level

Benchmarking on the ARM platform was conducted using an Ubuntu 24.04.3 LTS virtual machine on an Apple M4 chip with 16 GB unified memory.

Table A2. Raw Data on ARM Architecture for the Performance Evaluation at the Hybrid Key Exchange Level for All 50 Runs (Time in μs).

Test Run	X25519MLKEM768				SecP256r1MLKEM768				SecP384r1MLKEM1024			
	Encaps		Decaps		Encaps		Decaps		Encaps		Decaps	
	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA
Run 1	37.37	36.93	30.49	22.59	44.33	44.31	42.26	34.86	733.38	747.94	381.58	383.58
Run 2	37.75	36.87	30.57	22.70	44.40	44.25	42.27	34.74	744.63	734.57	388.96	370.83
Run 3	37.63	38.26	30.51	22.77	44.29	44.09	42.31	34.48	742.77	736.56	385.46	372.25
Run 4	37.73	37.33	30.66	25.78	44.29	44.36	42.20	34.66	742.67	736.45	387.35	371.61
Run 5	37.74	37.33	30.80	23.01	44.39	44.47	42.30	34.84	774.21	741.29	405.42	373.00
Run 6	37.98	37.59	30.63	23.32	44.32	44.65	42.28	34.85	742.86	761.04	390.08	389.78
Run 7	37.79	38.58	30.63	23.01	44.40	44.83	42.29	34.68	755.35	739.37	390.06	373.79
Run 8	37.76	37.46	30.74	23.09	44.31	44.65	42.42	34.75	744.42	749.75	386.50	375.33
Run 9	37.89	37.74	30.78	23.18	44.39	44.53	42.24	34.73	752.53	741.66	392.97	374.58
Run 10	38.08	37.67	31.01	23.11	44.16	44.73	42.34	34.87	768.25	745.53	401.67	375.42

Table A2. *Cont.*

Test Run	X25519MLKEM768				SecP256r1MLKEM768				SecP384r1MLKEM1024			
	Encaps		Decaps		Encaps		Decaps		Encaps		Decaps	
	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA	CCA	CPA
Run 11	39.11	38.10	30.80	23.17	44.33	44.55	42.47	35.13	742.94	744.97	385.65	374.72
Run 12	37.96	37.48	30.67	23.10	46.53	44.69	42.47	35.05	765.29	738.09	405.97	373.18
Run 13	38.08	37.57	30.64	23.15	44.37	47.02	42.28	34.77	748.32	743.68	393.80	376.36
Run 14	37.79	37.73	30.72	23.05	44.42	45.22	42.27	34.87	755.67	751.90	390.98	378.83
Run 15	37.85	37.96	30.87	26.87	44.39	45.11	42.66	35.11	748.69	777.72	387.90	394.89
Run 16	37.98	37.76	32.63	23.34	44.59	44.78	42.74	34.80	748.13	753.58	385.70	381.68
Run 17	38.02	37.51	30.80	26.70	44.28	44.47	42.33	35.07	771.01	740.74	408.64	375.58
Run 18	37.84	39.09	30.83	23.14	44.96	44.33	42.73	34.86	768.24	745.45	405.90	375.72
Run 19	38.00	37.59	30.70	23.17	44.57	44.60	42.52	34.92	749.25	761.01	389.02	382.30
Run 20	37.97	37.92	30.72	23.21	44.49	44.58	42.47	35.08	753.90	753.58	388.97	374.64
Run 21	37.88	37.75	30.71	23.22	44.50	44.64	42.35	34.84	794.07	744.79	418.24	374.26
Run 22	37.98	37.92	30.68	23.18	44.44	46.93	42.34	35.41	752.53	741.75	390.27	376.03
Run 23	37.96	37.66	30.81	23.26	44.29	44.99	42.36	35.25	754.15	738.09	394.89	373.32
Run 24	37.97	37.85	30.78	23.19	44.25	44.91	42.29	34.89	749.00	761.42	387.95	390.88
Run 25	37.91	37.76	30.80	23.26	44.24	45.47	42.46	35.12	778.61	754.53	404.55	385.85
Run 26	37.91	37.77	30.80	24.89	44.41	45.17	42.43	35.01	745.90	765.89	387.15	392.05
Run 27	37.90	37.76	30.81	23.22	44.43	51.36	42.32	35.51	748.13	742.39	386.90	373.83
Run 28	37.70	37.68	30.94	23.14	44.38	45.34	42.33	35.21	771.02	751.69	406.23	375.75
Run 29	37.82	37.71	33.25	23.19	44.62	44.91	43.08	35.17	741.20	745.16	385.95	376.03
Run 30	37.78	37.68	30.80	23.30	48.10	46.21	44.68	36.03	754.00	743.68	386.35	374.81
Run 31	37.89	37.78	30.84	23.23	44.52	45.69	42.50	35.48	752.45	744.42	389.12	374.81
Run 32	37.71	37.86	30.80	23.27	50.35	44.92	42.47	35.41	751.69	764.53	389.32	389.53
Run 33	47.16	37.81	30.81	23.22	44.42	45.35	42.38	35.40	751.31	743.13	388.16	372.86
Run 34	40.13	37.88	30.85	23.21	44.77	45.04	42.25	35.22	770.64	746.08	393.34	375.70
Run 35	37.74	37.90	30.87	23.17	44.20	44.99	46.11	35.28	744.60	744.79	386.60	374.36
Run 36	37.85	37.73	30.79	23.21	44.38	45.25	42.34	35.38	742.30	748.06	386.85	417.65
Run 37	38.02	37.81	30.83	23.23	44.37	45.09	42.36	35.33	752.26	764.31	390.78	393.34
Run 38	37.94	37.84	30.72	23.27	44.54	45.11	42.42	35.38	767.84	741.94	414.47	375.70
Run 39	38.07	37.70	30.78	24.78	44.35	44.74	42.52	35.06	755.05	742.39	392.98	376.06
Run 40	37.94	38.56	30.77	23.30	44.49	44.79	42.41	35.45	772.01	744.97	403.03	375.89
Run 41	38.03	38.23	30.87	23.32	44.28	44.59	42.39	35.06	753.86	750.56	392.29	375.80
Run 42	38.15	37.95	30.80	23.31	44.27	44.80	42.46	34.92	760.04	743.68	388.90	374.67
Run 43	37.93	37.90	30.85	23.27	44.27	44.61	42.41	34.87	756.58	740.56	391.62	372.62
Run 44	37.91	38.00	30.84	23.29	44.58	44.86	42.52	34.85	747.50	744.60	386.95	374.98
Run 45	37.88	37.86	30.60	23.29	44.61	44.45	42.40	34.87	750.63	742.39	389.54	374.30
Run 46	37.92	37.76	30.95	23.35	44.40	44.64	42.34	34.92	748.26	744.05	384.76	374.53
Run 47	37.98	37.75	30.83	23.36	44.36	44.60	42.42	34.87	746.08	747.01	385.60	376.18
Run 48	37.89	37.95	30.80	23.33	44.44	44.73	42.36	35.05	744.68	742.21	383.58	373.00
Run 49	37.81	37.70	30.76	23.30	44.52	44.94	42.37	34.83	743.68	747.31	384.57	377.00
Run 50	37.78	37.87	30.96	23.55	44.45	44.59	42.47	35.03	745.26	752.26	386.80	374.36
Mean	38.14	37.80	30.86	23.45	44.65	45.04	42.53	35.04	753.96	747.47	392.21	378.29
SD	1.36	0.35	0.45	0.85	1.03	1.08	0.63	0.28	11.74	8.70	8.42	8.29
95%CI(LB)	37.75	37.70	30.73	23.21	44.36	44.73	42.35	34.96	750.62	745.00	389.81	375.93
95%CI(UB)	38.52	37.90	30.98	23.70	44.94	45.34	42.71	35.12	757.29	749.94	394.60	380.64

Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Apple M4 chip, 16 GB unified memory.

Appendix B. Raw Data for the Performance Evaluation at the Protocol Level

We employed two different processor architectures for the protocol level experiments: an Intel Core i7-1165G7 CPU (x86-64) and an Apple M4 chip (ARM).

Appendix B.1. Raw Data on x86-64 Architecture at the Protocol Level

The x86-64 benchmark configuration comprised an Ubuntu 24.04.3 LTS virtual machine equipped with an Intel Core i7-1165G7 CPU and 16 GB of RAM.

Table A3. Raw Data on x86-64 Architecture for the Performance Evaluation at the Protocol Level for All 50 Runs (Time in µs).

Test Run	X25519MLKEM768		SecP256r1MLKEM768		SecP384r1MLKEM1024	
	CCA	CPA	CCA	CPA	CCA	CPA
Run 1	620.44	695.42	862.07	915.49	3071.16	2901.23
Run 2	724.07	667.24	928.14	1000.00	3311.04	2712.93
Run 3	642.86	663.79	844.77	993.38	2811.50	3065.02
Run 4	681.62	668.95	902.56	1016.22	3049.18	3031.25
Run 5	734.30	638.84	966.08	884.04	2924.53	2912.62

Table A3. *Cont.*

Test Run	X25519MLKEM768		SecP256r1MLKEM768		SecP384r1MLKEM1024	
	CCA	CPA	CCA	CPA	CCA	CPA
Run 6	796.81	694.09	910.93	924.17	3244.15	3000.00
Run 7	689.66	705.47	939.73	900.11	3288.14	2959.50
Run 8	748.82	728.42	906.49	866.94	3234.32	2904.19
Run 9	727.45	700.35	961.74	1005.08	3081.97	2915.36
Run 10	691.36	650.62	925.34	935.55	3092.11	2896.34
Run 11	679.86	680.27	1078.84	859.21	3235.29	3047.62
Run 12	658.98	604.44	996.92	925.53	3407.64	2878.79
Run 13	794.39	591.13	1120.19	903.36	3181.82	2984.62
Run 14	893.97	639.00	1079.06	909.09	3168.32	3015.38
Run 15	825.29	681.62	884.21	830.04	3414.63	2943.04
Run 16	713.06	644.10	924.09	929.07	3277.59	3250.77
Run 17	687.83	758.75	1006.86	863.16	3074.43	3049.18
Run 18	790.66	615.64	921.61	845.62	3037.97	3177.57
Run 19	758.62	709.52	921.18	868.20	3019.48	3134.80
Run 20	716.85	692.90	936.90	926.32	3215.43	2690.06
Run 21	703.13	698.47	1029.72	989.01	3192.18	2943.04
Run 22	717.45	681.82	961.12	939.81	3067.09	2946.71
Run 23	713.06	712.42	917.62	905.22	3121.02	2893.89
Run 24	700.09	712.45	980.60	884.15	3112.58	3203.88
Run 25	663.13	671.77	1082.73	922.92	3040.75	3015.87
Run 26	686.81	687.29	1053.20	836.73	2981.37	2984.13
Run 27	728.97	700.98	927.42	1093.17	3290.32	2944.98
Run 28	782.18	652.95	961.34	896.09	2664.58	3278.15
Run 29	731.93	666.67	900.72	913.87	3265.99	3476.82
Run 30	750.45	683.01	848.26	881.76	3069.31	3044.87
Run 31	659.63	630.25	987.78	849.34	3164.56	3149.35
Run 32	740.74	703.05	994.59	938.82	3233.33	2830.77
Run 33	682.01	769.23	885.42	892.12	3202.61	2943.04
Run 34	664.24	631.23	926.83	898.99	3094.46	2919.25
Run 35	774.25	707.16	989.43	810.81	3071.90	2974.68
Run 36	831.68	660.87	1002.18	832.54	3225.81	3535.35
Run 37	779.82	750.72	856.55	933.63	3084.42	2971.25
Run 38	670.84	651.24	904.11	789.20	3344.26	3110.37
Run 39	754.04	723.51	931.22	826.99	3169.93	3187.92
Run 40	634.63	721.75	1094.09	907.08	3003.10	3218.39
Run 41	783.37	666.09	1011.24	898.64	3197.49	2871.29
Run 42	745.50	818.60	878.50	853.91	2852.76	2839.12
Run 43	730.24	744.39	879.85	856.24	3202.61	3034.06
Run 44	701.28	669.49	913.65	962.80	3123.03	3079.37
Run 45	663.13	649.02	884.35	885.31	3130.99	2821.32
Run 46	743.06	722.37	962.73	910.99	3190.79	3127.04
Run 47	735.03	784.12	1081.38	847.64	3193.55	2980.77
Run 48	641.57	676.03	953.39	997.85	2993.63	2774.39
Run 49	696.20	675.79	939.92	976.95	3072.10	3049.18
Run 50	705.13	760.87	899.62	948.48	3247.59	3019.48
Mean	721.81	688.28	953.15	907.63	3134.90	3012.78
SD	55.39	45.88	69.11	59.67	142.41	164.29
95% CI	[706.07, 737.55]	[675.24, 701.32]	[933.51, 972.79]	[890.67, 924.59]	[3094.42, 3175.37]	[2966.09, 3059.47]

Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Intel Core i7-1165G7 CPU, 16 GB RAM.

Appendix B.2. Raw Data on ARM Architecture at the Protocol Level

Benchmarking on the ARM platform was conducted using an Ubuntu 24.04.3 LTS virtual machine on an Apple M4 chip with 16 GB unified memory.

Table A4. Raw Data on ARM Architecture for the Performance Evaluation at the Protocol Level for All 50 Runs (Time in μs).

Test Run	X25519MLKEM768		SecP256r1MLKEM768		SecP384r1MLKEM1024	
	CCA	CPA	CCA	CPA	CCA	CPA
Run 1	173.36	160.78	229.30	226.85	963.42	952.38
Run 2	166.28	161.37	230.18	219.74	960.80	939.12
Run 3	167.53	156.82	227.05	224.79	960.80	944.99
Run 4	173.35	160.83	226.87	223.04	962.64	940.45
Run 5	163.47	157.48	226.75	223.35	972.97	946.07
Run 6	165.54	157.06	228.25	221.60	955.14	942.06
Run 7	171.28	159.91	227.35	221.96	974.53	933.08
Run 8	173.06	157.44	230.80	223.23	963.33	933.08
Run 9	167.39	160.58	226.95	218.41	956.94	944.29
Run 10	172.22	160.97	231.76	222.29	953.74	948.32
Run 11	171.34	160.12	229.37	216.66	959.89	944.11
Run 12	166.69	161.30	233.33	218.14	950.79	933.96
Run 13	170.95	163.44	240.28	225.73	960.34	940.29

Table A4. *Cont.*

Test Run	X25519MLKEM768		SecP256r1MLKEM768		SecP384r1MLKEM1024	
	CCA	CPA	CCA	CPA	CCA	CPA
Run 14	169.35	161.44	237.58	223.09	959.89	942.51
Run 15	173.96	162.74	232.77	222.30	955.11	947.22
Run 16	176.52	163.73	233.66	219.21	956.94	945.18
Run 17	170.48	161.37	230.65	221.11	959.23	939.57
Run 18	177.52	161.24	229.92	220.17	951.24	955.08
Run 19	174.51	158.68	230.19	221.67	954.20	949.46
Run 20	173.87	162.33	232.94	220.76	952.38	937.79
Run 21	180.98	163.47	229.83	222.08	950.79	946.07
Run 22	173.38	175.75	226.82	223.18	956.02	946.52
Run 23	176.85	162.64	227.83	218.19	951.24	933.52
Run 24	177.90	163.10	227.37	219.70	953.29	940.01
Run 25	174.97	177.02	227.65	224.84	953.29	952.16
Run 26	172.93	164.52	231.99	219.63	960.80	934.84
Run 27	182.24	162.78	227.72	218.92	954.20	947.67
Run 28	171.00	161.53	230.91	221.40	959.89	945.63
Run 29	174.89	166.49	228.53	217.91	953.74	947.42
Run 30	178.70	160.64	226.42	221.57	958.06	938.68
Run 31	173.19	165.11	226.09	222.03	959.89	934.40
Run 32	177.56	164.95	228.43	219.87	947.62	940.29
Run 33	182.24	163.83	224.87	220.05	949.43	936.03
Run 34	173.87	165.69	225.05	222.95	959.89	944.73
Run 35	173.74	164.24	225.67	218.67	951.93	938.24
Run 36	183.02	163.56	227.78	221.23	954.20	937.79
Run 37	173.91	164.75	228.75	222.15	949.88	942.06
Run 38	175.87	165.52	228.55	222.42	953.74	941.62
Run 39	177.21	171.36	228.82	223.06	964.03	948.56
Run 40	175.38	166.69	230.30	222.63	957.40	940.29
Run 41	178.06	164.91	230.01	223.11	957.40	942.51
Run 42	175.78	167.59	230.47	222.49	960.34	951.25
Run 43	182.27	165.35	226.92	221.22	943.76	942.51
Run 44	180.83	165.01	228.25	217.89	961.72	946.77
Run 45	178.69	164.14	229.43	221.19	951.93	945.18
Run 46	182.87	167.94	227.87	220.44	952.38	939.12
Run 47	182.96	163.34	233.33	218.28	958.51	934.40
Run 48	176.13	163.66	226.67	222.49	958.77	938.24
Run 49	177.75	163.76	228.96	221.81	952.61	939.12
Run 50	180.51	166.03	235.72	220.68	968.82	942.06
Mean	174.89	163.50	229.46	221.32	957.00	942.33
SD	4.85	3.93	3.08	2.15	5.93	5.43
95% CI	[173.51, 176.26]	[162.38, 164.62]	[228.58, 230.34]	[220.71, 221.93]	[955.31, 958.68]	[940.79, 943.88]

Experimental environment: Ubuntu 24.04.3 LTS virtual machine, Apple M4 chip, 16 GB unified memory.

References

1. Alnahawi, N.; Müller, J.; Oupický, J.; Wiesmaier, A. A Comprehensive Survey on Post-Quantum TLS. *IACR Commun. Cryptol.* **2024**, *1*, 6. [CrossRef]
2. Shor, P.W. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In Proceedings of the 35th Annual Symposium on Foundations of Computer Science, Santa Fe, NM, USA, 20–22 November 1994; pp. 124–134.
3. Diffie, W.; Hellman, M.E. New directions in cryptography. *IEEE Trans. Inf. Theory* **1976**, *22*, 644–654. [CrossRef]
4. Rivest, R.L.; Shamir, A.; Adleman, L.M. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. *Commun. ACM* **1978**, *21*, 120–126. [CrossRef]
5. Rescorla, E. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. 2018. Available online: https://www.rfc-editor.org/info/rfc8446 (accessed on 23 October 2025).
6. Miller, V.S. Use of Elliptic Curves in Cryptography. In Proceedings of the Advances in Cryptology—CRYPTO '85, Santa Barbara, CA, USA, 18–22 August 1985; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 1985; Volume 218, pp. 417–426.
7. Paquin, C.; Stebila, D.; Tamvada, G. Benchmarking Post-quantum Cryptography in TLS. In Proceedings of the Post-Quantum Cryptography—11th International Conference, PQCrypto 2020, Paris, France, 15–17 April 2020; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2020; Volume 12100, pp. 72–91.
8. Dowling, B.; Fischlin, M.; Günther, F.; Stebila, D. A Cryptographic Analysis of the TLS 1.3 Handshake Protocol. *J. Cryptol.* **2021**, *34*, 37. [CrossRef]
9. Schwabe, P.; Stebila, D.; Wiggers, T. Post-Quantum TLS Without Handshake Signatures. In Proceedings of the CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, 9–13 November 2020; ACM: New York, NY, USA, 2020; pp. 1461–1480.

10. Huguenin-Dumittan, L.; Vaudenay, S. On IND-qCCA Security in the ROM and Its Applications—CPA Security Is Sufficient for TLS 1.3. In Proceedings of the Advances in Cryptology—EUROCRYPT 2022—41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, 30 May–3 June 2022; Part III; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2022; Volume 13277, pp. 613–642.
11. Zhou, B.; Jiang, H.; Zhao, Y. CPA-Secure KEMs are also Sufficient for Post-quantum TLS 1.3. In Proceedings of the Advances in Cryptology—ASIACRYPT 2024—30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, 9–13 December 2024; Part III; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2024; Volume 15486, pp. 433–464.
12. Chen, J.; Zhou, B.; Chen, R.; Jiang, H.; Wang, Y.; Huang, X.; Zhao, Y.; Yung, M. Post-Quantum TLS 1.3 Handshake from CPA-Secure KEMs with Tighter Reductions. *Cryptol. ePrint Arch.* **2025**, Paper 2025/1748. Available online: https://eprint.iacr.org/2025/1748 (accessed on 23 October 2025).
13. National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*; Technical Report Federal Information Processing Standards Publications (FIPS) 203; U.S. Department of Commerce: Washington, DC, USA, 2024. Available online: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf (accessed on 23 October 2025).
14. National Institute of Standards and Technology. NIST Selects HQC as Fifth Algorithm for Post-Quantum Encryption. 2025. Available online: https://www.nist.gov/news-events/news/2025/03/nist-selects-hqc-fifth-algorithm-post-quantum-encryption (accessed on 23 October 2025).
15. Hofheinz, D.; Hövelmanns, K.; Kiltz, E. A Modular Analysis of the Fujisaki-Okamoto Transformation. In Proceedings of the Theory of Cryptography—15th International Conference, TCC 2017, Baltimore, MD, USA, 12–15 November2017; Part I; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2017; Volume 10677, pp. 341–371.
16. Guo, Q.; Johansson, T.; Nilsson, A. A Key-Recovery Timing Attack on Post-quantum Primitives Using the Fujisaki-Okamoto Transformation and Its Application on FrodoKEM. In Proceedings of the Advances in Cryptology—CRYPTO 2020—40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, 17–21 August 2020; Part II; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2020; Volume 12171, pp. 359–386.
17. Hövelmanns, K.; Hülsing, A.; Majenz, C.; Sisinni, F. (Un)breakable Curses—Re-encryption in the Fujisaki-Okamoto Transform. In Proceedings of the Advances in Cryptology—EUROCRYPT 2025—44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, 4–8 May 2025; Part II; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2025; Volume 15602, pp. 245–274.
18. Bellare, M.; Rogaway, P. Random Oracles are Practical: A Paradigm for Designing Efficient Protocols. In Proceedings of the CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security, Fairfax, VA, USA, 3–5 November 1993; ACM: New York, NY, USA, 1993; pp. 62–73.
19. Boneh, D.; Dagdelen, Ö.; Fischlin, M.; Lehmann, A.; Schaffner, C.; Zhandry, M. Random Oracles in a Quantum World. In Proceedings of the Advances in Cryptology—ASIACRYPT 2011—17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, Republic of Korea, 4–8 December 2011; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2011; Volume 7073, pp. 41–69.
20. Benčina, B.; Dowling, B.; Maram, V.; Xagawa, K. Post-Quantum Cryptographic Analysis of SSH. In Proceedings of the IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, 12–15 May 2025; pp. 595–613.
21. Kwiatkowski, K.; Kampanakis, P.; Westerbaan, B.; Stebila, D. Post-Quantum Hybrid ECDHE-MLKEM Key Agreement for TLSv1.3. Internet-Draft draft-ietf-tls-ecdhe-mlkem-01, Internet Engineering Task Force. 2025. Work in Progress. Available online: https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/ (accessed on 23 October 2025).
22. Stebila, D.; Fluhrer, S.; Gueron, S. Hybrid Key Exchange in TLS 1.3. Internet-Draft draft-ietf-tls-hybrid-design-16, Internet Engineering Task Force. 2025. Work in Progress. Available online: https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/16/ (accessed on 23 October 2025).
23. Westerbaan, B. X (Twitter) Post. 2024. Available online: https://x.com/bwesterb/status/1771958142147973390 (accessed on 23 October 2025).
24. Jiang, H.; Ma, Z.; Zhang, Z. Post-quantum Security of Key Encapsulation Mechanism Against CCA Attacks with a Single Decapsulation Query. In Proceedings of the Advances in Cryptology—ASIACRYPT 2023—29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, 4–8 December 2023; Part IV; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2023; Volume 14441, pp. 434–468.
25. Arfaoui, G.; Bultel, X.; Fouque, P.; Nedelcu, A.; Onete, C. The privacy of the TLS 1.3 protocol. *Proc. Priv. Enhancing Technol.* **2019**, *2019*, 190–210. [CrossRef]
26. Diemert, D.; Jager, T. On the Tight Security of TLS 1.3: Theoretically Sound Cryptographic Parameters for Real-World Deployments. *J. Cryptol.* **2021**, *34*, 30. [CrossRef]

27. Davis, H.; Günther, F. Tighter Proofs for the SIGMA and TLS 1.3 Key Exchange Protocols. In Proceedings of the Applied Cryptography and Network Security—19th International Conference, ACNS 2021, Kamakura, Japan, 21–24 June 2021; Part II; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2021; Volume 12727, pp. 448–479.
28. Dowling, B.; Fischlin, M.; Günther, F.; Stebila, D. A Cryptographic Analysis of the TLS 1.3 Handshake Protocol Candidates. In Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, 12–16 October 2015; ACM: New York, NY, USA, 2015; pp. 1197–1210.
29. Dowling, B.; Fischlin, M.; Günther, F.; Stebila, D. A Cryptographic Analysis of the TLS 1.3 draft-10 Full and Pre-shared Key Handshake Protocol. *Cryptol. ePrint Arch.* **2016**, Paper 2016/081. Available online: https://eprint.iacr.org/2016/081 (accessed on 23 October 2025).
30. Dowling, B. Provable Security of Internet Protocols. Ph.D. Thesis, Queensland University of Technology, Brisbane, Australia, 2017.
31. Fischlin, M.; Günther, F. Replay Attacks on Zero Round-Trip Time: The Case of the TLS 1.3 Handshake Candidates. In Proceedings of the 2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, 26–28 April 2017; pp. 60–75.
32. Günther, F. Modeling Advanced Security Aspects of Key Exchange and Secure Channel Protocols. Ph.D. Thesis, Technische Universität Darmstadt, Darmstadt, Germany, 2018.
33. Brendel, J.; Fischlin, M.; Günther, F.; Janson, C. PRF-ODH: Relations, Instantiations, and Impossibility Results. In Proceedings of the Advances in Cryptology—CRYPTO 2017—37th Annual International Cryptology Conference, Santa Barbara, CA, USA, 20–24 August 2017; Part III; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2017; Volume 10403, pp. 651–681.
34. Giacon, F.; Heuer, F.; Poettering, B. KEM Combiners. In Proceedings of the Public-Key Cryptography—PKC 2018—21st IACR International Conference on Practice and Theory of Public-Key Cryptography, Rio de Janeiro, Brazil, 25–29 March 2018; Part I; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2018; Volume 10769, pp. 190–218.
35. Bindel, N.; Brendel, J.; Fischlin, M.; Goncalves, B.; Stebila, D. Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange. In Proceedings of the Post-Quantum Cryptography—10th International Conference, PQCrypto 2019, Chongqing, China, 8–10 May 2019; Revised Selected Papers; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2019; Volume 11505, pp. 206–226.
36. Barbosa, M.; Connolly, D.; Duarte, J.D.; Kaiser, A.; Schwabe, P.; Varner, K.; Westerbaan, B. X-Wing. *IACR Commun. Cryptol.* **2024**, *1*, 21. [CrossRef]
37. Liu, Y.; Zhou, B.; Jiang, H. CuKEM: A Concise and Unified Hybrid Key Encapsulation Mechanism. In Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, 13–17 October 2025; ACM: New York, NY, USA, 2025; pp. 4707–4721.
38. Zheng, J.; Zhu, H.; Dong, Y.; Song, Z.; Zhang, Z.; Yang, Y.; Zhao, Y. Faster Post-quantum TLS 1.3 Based on ML-KEM: Implementation and Assessment. In Proceedings of the Computer Security—ESORICS 2024—29th European Symposium on Research in Computer Security, Bydgoszcz, Poland, 16–20 September 2024; Part II; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2024; Volume 14983, pp. 123–143.
39. Nielsen, M.A.; Chuang, I.L. *Quantum Computation and Quantum Information*, 10th Anniversary ed.; Cambridge University Press: Cambridge, UK, 2016.
40. Bindel, N.; Hamburg, M.; Hövelmanns, K.; Hülsing, A.; Persichetti, E. Tighter Proofs of CCA Security in the Quantum Random Oracle Model. In Proceedings of the Theory of Cryptography—17th International Conference, TCC 2019, Nuremberg, Germany, 1–5 December 2019; Part II; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2019; Volume 11892, pp. 61–90.
41. Boneh, D.; Shoup, V. *A Graduate Course in Applied Cryptography (ver. 0.6)*; 2023. Available online: https://toc.cryptobook.us/ (accessed on 23 October 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.